algorithms

MDPI

Article

Density-Adaptive Rank Space Neighbor Extraction from Multidimensional Data Using Boundary-Corrected Hyper Windows

Mohsin Shahzad [1,*], Muhammad Amjad [1], Muhammad Bux Alvi [2], Serosh Karim Noon [3,*], Kashif Hussain Memon [4] and Fareeha Zahoor [5]

[1] Department of Electrical Engineering, The Islamia University of Bahawalpur, Bahawalpur 63100, Pakistan; muhammad.amjad@iub.edu.pk
[2] Department of Computer Systems Engineering, The Islamia University of Bahawalpur, Bahawalpur 63100, Pakistan; mbalvi@iub.edu.pk
[3] Department of Electrical Engineering, NFC Institute of Engineering & Technology, Multan 60000, Pakistan
[4] Lero, University of Limerick, V94 Limerick, Ireland; kashif.memon@ul.ie
[5] Department of Computer Science, The Islamia University of Bahawalpur, Bahawalpur 63100, Pakistan; fareeha_zahoor@hotmail.com
* Correspondence: engr.mohsins@gmail.com (M.S.); seroshkarim@nfciet.edu.pk (S.K.N.)

Abstract

k-nearest-neighbor (k-NN) methods remain fundamental to pattern recognition, classification, and image analysis because they are simple, non-parametric, and effective for nonlinear decision structures. However, k-NN methods are constrained by two related limitations: explicit pairwise distance computation and reliance on fixed neighborhood parameters, most notably the parameter *k* in k-NN and the fixed window-size parameter *Diff* in Fast Geometrical Extraction of Nearest Neighbors (FGENN). FGENN introduced a distance-free method by transforming data into rank space and extracting nearest neighbors through a geometric hyper-window rather than metric-distance ordering, but its fixed window size does not adapt to local density variation, causing over-selection in dense regions and under-selection in sparse or boundary regions. In this paper, we propose Density-Adaptive Rank-Space Neighbor Extraction using Boundary-Corrected Hyper-windows (DARNE), an adaptive rank-space neighbor-extraction and geometric neighborhood-filtering mechanism that generalizes FGENN, where its fixed window and exhaustive sweep are replaced using an adaptive window size governed by a two-stage local-to-global occupancy-density control law. DARNE outputs ordinal rank neighborhoods in two steps: Stage 1 checks the local rank-lattice occupancy of the query neighborhood by using boundary-corrected hyper-window volumes; Stage 2 analytically scales the window, based on the local-to-global density ratio, without calculating pairwise Euclidean distances. DARNE also outperforms fixed-window FGENN in query latency for all benchmarks and brute-force k-NN variants on the IRIS, SONAR and Covertype datasets, achieving speedups of 2.78×, 11.55× and 21.14×. DARNE achieved accuracies of 97.78%, 94.09%, and 69.12% on the IRIS, SONAR and Covertype datasets, respectively, at a lower computational cost for queries relative to existing methods. Together, these results demonstrate that two-stage density-adaptive windowing in rank space is a reproducible and computationally scalable neighborhood-filtering framework for a wide range of multi-dimensional classification and hashing benchmarks.

Keywords: nearest-neighbor search; rank-space transformation; adaptive window selection; density-aware classification; *k*-nearest neighbors; geometric neighbor extraction

Check for updates

Academic Editor: Igor Aizenberg

Received: 2 October 2026
Revised: 29 October 2026
Accepted: 30 October 2026
Published: 2 November 2026

Copyright: © 2026 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license.

1. Introduction

Nearest-neighbor learning is one of the most durable paradigms in machine learning and pattern recognition. Its core principle is that a query object can be classified, clustered, or retrieved by examining the objects that are closest to it in a chosen representation space [1–3]. k-NN and its variants are used in classification, regression, clustering, vector quantization, retrieval, hashing, text analysis, biomedical decision support, and image processing [4–10]. The strength of these methods lies in their simplicity and locality: rather than learning a global parametric model, they infer decisions from nearby examples [11,12]. For example, CNN-derived features paired with k-NN have recently reported an AUC of 0.996 in glioma grade classification and near-perfect accuracy in breast cancer mammogram diagnosis, illustrating how representation quality can substantially amplify k-NN's discriminative power [13,14]. Beyond biomedical imaging, multi-dimensional data fusion, unsupervised clustering, and local window-based anomaly or prospectivity modeling are central to geoscientific and remote-sensing workflows. Tree-based anomaly detectors, notably Isolation Forest and its extension, Extended Isolation Forest, illustrate how bias introduced by axis-parallel partitioning and sensitivity to hyperparameter choice can compromise target identification in complex, high-dimensional feature spaces [15]. Complementing this, hybrid fusion frameworks that integrate K-means clustering, local window singularity contrast, and Fuzzy-AHP weighting highlight the benefit of adaptive local thresholding and fuzzy decision rules for reconciling heterogeneous data layers [16]. The Neuro-Fuzzy-AHP (NFAHP) method integrates alteration maps of ASTER with geological data sets through Fuzzy-AHP and a back-propagation neural network to assess mineral exploration factors [17].

The same locality that makes nearest-neighbor methods effective also creates computational and methodological difficulties. For a dataset $D = \{x_i\}_{i=1}^{N}, x_i \in \mathbb{R}^M$, a conventional exact k-NN search evaluates a distance $dist(x_q,x_i)$ between the query x_q and every database point x_i. Even before sorting or selecting the nearest k neighbors, the method requires $O(N.M)$ arithmetic operations per query under a naive implementation. In large, high-dimensional, streaming, or image-patch settings, repeated distance evaluation becomes a major computational bottleneck [5,18].

A second limitation is the rigidity of fixed neighborhood control. Classical k-NN uses a single k for all query points [10]. A small k produces unreliable results in sparse areas and near class boundaries, while a large k over-smooths dense clusters. Weighted k-NN [19], adaptive k-NN [11], fuzzy k-NN [20], condensed k-NN [21], and related variants improve particular aspects of decision quality, yet they usually retain explicit distance evaluation and introduce additional weighting or heuristic parameters.

The FGENN method [18] represents a different line of work. Instead of ordering points by metric distance, FGENN first ranks feature values independently and then extracts neighbors by applying a fixed-size geometric hyper-window in rank space. This eliminates explicit pairwise distance computation during neighbor extraction and provides a fast mechanism for identifying a variable number of neighbors. However, FGENN relies on a *Diff* (window-size) parameter that controls the size of the geometric window; the original method does not provide a formal local density rule for selecting this window size per query point [22,23]. Closing this gap is the central objective of the present work. In addition, the number of iterations for window expansion is linear in the preset *Diff* value (e.g., 30 or 100 iterations per query) in FGENN, and the proximity between two values expressed by ordinal rank is not the same as the proximity expressed by the Euclidean distance, which means that if the window is set to a fixed rank, the true metric proximity may be neglected in a sparse boundary region and too many false neighbors may be included in a dense region.

This paper proposes Density-Adaptive Rank space Neighbor Extraction using Boundary-Corrected Hyper-windows (DARNE). To avoid overstating its scope, two fundamental points are made initially: (1) DARNE is not an exact metric nearest-neighbor search algorithm; it retrieves ordinal neighbor points (rank-neighbors) within axis-aligned hyper-windows in dense rank space, which can be used in decision rules (classification and hashing) but can also optionally be followed by a metric-verification step over the filtered neighbor set. The specific innovation of the DARNE over FGENN [18] is the use of boundary-corrected local-to-global rank-lattice occupancy control law instead of the globally fixed control-window parameter (*Diff*) and exhaustive *Diff*-step iterative sweep of FGENN, which adapts the hyper-window size per query in a two-stage ($T = 2$) direct update. Moreover, DARNE provides a query time advantage over $O(NM)$ Brute-force Distance Evaluation only when applied to large tabular datasets. For low-dimensional metric spaces ($M \leq 10$), classical space partitioning structures like KD-tree [24] and Ball-tree [25] are highly competitive compared to the multi-iteration, fixed-window sweep of FGENN. The remainder of this paper is organized as follows: Section 2 presents related work within five methodological families. In Section 3, DARNE is formalized along with its rank-space transformation, rank/metric divergence analysis, two-stage adaptive window-selection rule, naive and indexed retrieval strategies, and complete research workflow. In Section 4, mathematical properties (monotonicity, scale invariance and finite termination), formal structural properties and the computation of *Big-O* complexity for DARNE are presented. In Section 5, the experimental methodology is detailed, which comprises the justification of the selection of the baselines, the full tables of hyper-parameter configurations, sensitivity analysis and the specifications of the benchmarking hardware. Section 6 presents experimental results for IRIS, SONAR and Covertype: Exact-k-NN Recall, and corrected speedups in runtime, as well as 95% confidence intervals, paired hypothesis tests, and k-NNH hashing. A separate discussion of strengths, weaknesses, rank-versus-metric trade-offs and baseline comparisons is provided in Section 7. Section 8 concludes the paper.

2. Related Work

Related work relevant to DARNE can be organized into five methodological families: classical distance-based nearest-neighbor methods [10]; weighted, fuzzy, evidential, and adaptive k-NN variants [8,11,26]; data-reduction, clustering-assisted, and graph-based methods [21,27]; approximate and indexed nearest-neighbor search [28,29]; and rank-space or geometric neighbor extraction [18,30,31]. This taxonomy is used because DARNE is not merely a classifier; it is a neighbor-extraction mechanism that can be embedded in classification, retrieval, hashing, and image-processing pipelines.

2.1. Classical Distance-Based Methods

The classical nearest-neighbor rule [10] and its k-NN generalization remain the reference point for the field. Numerous extensions accelerate large-scale search using clustering-based neighbor pruning [1], neural-code representations [1], KD-trees [24], and ball-trees [25], while k-NN-join methods use block-nested-loop partitioning with early termination to reduce all-pairs evaluation cost [32]. A utility-maximizing formulation derived the optimal k as a function of the class-conditional density ratio and the false-positive/false-negative cost ratio, with applications to credit scoring and medical triage [33]. All of these methods, however, retain explicit pairwise distance computation as their core operation. Diffusion-based anomaly detectors have further shown that such distance measures lose discriminatory power in nonlinear, high-dimensional regions [34]. Exact spatial indexing trees like KD-trees [24] and ball-trees [25] can lower the average complexity of queries per problem from $O(NM)$ to $O(M \log N)$, at the cost of an $O(MN \log N)$ tree construction phase,

in low-dimension ($M \leq 10$) spaces. But, for high dimensionality (e.g., $M \geq 50$ in SONAR and Covertype), the dimensionality problem makes the efficiency of bounding-ball and hyperplane pruning drop quickly, returning the tree search to brute-force, $O(NM)$ cost.

2.2. Weighted, Fuzzy, Evidential, and Adaptive k-Nearest Neighbor Methods

Weighted k-NN [19] assigns larger influence to closer neighbors; fuzzy k-NN [20] distributes class membership instead of hard labels; evidential k-NN introduces uncertainty through the Dempster–Shafer theory [26]; adaptive k-NN estimates a local or query-specific k [11,12]. A locally scaled similarity kernel has likewise been shown to match tuned k-NN accuracy without dataset-specific metric tuning [35]. Recent variants jointly optimize vote weights and neighborhood size from local density feedback [36], combine exponential distance decay with a per-neighbor validity coefficient, or iteratively reweight misclassified samples to counter class imbalance [37]. Multi-distance fusion approaches combine several distance kernels with learnable weights and density-adjusted k [38], and picture-fuzzy and fuzzy-rough extensions enrich uncertainty representation for tabular and text data [39,40]. Evidential frameworks integrate neighborhood search and feature weighting within a single Dempster–Shafer combination stage, including distributed variants for large datasets [41,42]. These variants improve decision rules but, like classical k-NN, retain explicit distance computation.

2.3. Data Reduction, Clustering, and Graph-Based Methods

Condensed nearest-neighbor methods reduce the number of stored references [21]; clustering-assisted k-NN restricts search to relevant clusters [12]; and graph-based approaches refine neighbor graphs for downstream tasks such as spectral clustering [27]. Prototype-selection techniques based on relative density and density-peak clustering separate harmful sample removal from prototype selection [43,44]. Hashing-based neighbor reduction lowers prediction latency with only minor accuracy loss for moderate intrinsic dimensionality [45]. Split-merge and shared-nearest-neighbor clustering algorithms use the k-NN graph itself as the clustering structure [46,47] and adaptive k-NN graph models dynamically adjust graph degree and edge weight from local density [48]. These methods improve scalability, but their construction and query phases still rely on distance measures.

2.4. Approximate and Index-Based Neighbor Search

Approximate methods like locality-sensitive hashing, graph-based ANN, and vector-indexing structures reduce retrieval time by accepting approximate results or using specialized data structures [29]. HNSW [49] remains the dominant navigable small-world graph; post-construction reordering (Gorder, RCM) reduces cache misses and cuts query latency by up to 40% at high recall [50], and empirical analysis suggests HNSW's hierarchical layers primarily create routing hubs rather than genuine multi-scale structure [51]. Multi-attribute range-filtering ANN combines proximity-graph traversal with attribute range pruning [52]; hardness-detection frameworks redirect difficult queries to a denser local search budget [53]; and learned or probabilistic routing policies reduce distance computations per query without modifying the underlying graph [54]. These methods are fast but remain metric/similarity-dependent and often approximate.

2.5. Rank-Space and Geometric Neighbor Extraction Methods

The idea of rank-space, distance-free neighbor retrieval traces back to the Rank Cover Tree [30], which prunes neighbors using only similarity ranks, and to comparison-based search, which relies solely on ordinal statements of the form "is point i closer to point j than to point k?" [31]. FGENN [18] advanced this direction by performing neighbor

extraction as a geometric operation in rank space: a data ranking algorithm ranks each feature independently to build a rank matrix, and a fixed-width hyper-window then cuts each ranked feature list to retrieve neighbors. FGENN reports improved time efficiency and competitive classification/hashing behavior, but its hyper-window is not selected by a formal local-density rule, and the number of returned neighbors cannot be precisely specified or predicted. Comparison-tree structures built by recursive space decomposition around random pivots achieve logarithmic height trees with formal failure-probability bounds [31], showing that neighborhood structure can be reconstructed from rank information alone. Table 1 summarizes the neighbor-control mechanism and distance dependence of each family and the limitation that DARNE targets.

Table 1. Comparison of neighbor-control mechanisms across representative methodological families.

Method Family	Neighbor-Control Mechanism	Distance Dependence	Main Limitation
Classical k-NN [10]	Fixed global k	High	Distance cost and fixed-k rigidity
Weighted [19]/Fuzzy k-NN [20,38]	Fixed k with weights/memberships	High	Improves decision rule, not distance-free
Adaptive k-NN [11,12,35,36]	Query-specific k (heuristic/learned)	High	Adaptive, but still distance-based
Clustering/ Condensed [21]	Reduced neighbor set	Med.–High	Preprocessing still relies on distances
Approximate/Indexed [28,49]	Index structure + approximation	Med.–High	Similarity-dependent, often approximate
FGENN [18]	Fixed-*Diff* rank-space hyper-window	None	*Diff* not density- adaptive
DARNE	Two-stage density-adaptive rank-space hyper-window	None	Retrieves rank-neighbors (not exact metric k-NN); $O(MN \log N)$ rank-sorting preprocessing

3. Proposed Method: DARNE

The research problem addressed here is to design, analyze, and validate a nearest-neighbor extraction framework that performs the following:

1. Avoids explicit pairwise metric distance computation.
2. Adapts its rank-space neighborhood size to local density variation.
3. Remains computationally efficient under a defensible implementation.
4. Preserves or improves predictive and retrieval quality across heterogeneous, high-dimensional, and image-based data.

DARNE addresses this by making the search window query specific and density aware, replacing distance computations with rank comparisons and interval intersections.

DARNE operates in two phases: a preprocessing phase that transforms the dataset into a rank-space representation and builds optional rank indexes (Algorithm 1) and a query phase that extracts a local neighbor set by adaptively growing or shrinking a hyper-window around the query point in rank space (Algorithm 2). DARNE avoids explicit pairwise metric distance computations during extraction; it still performs rank comparisons, interval searches, and neighbor intersections. The precise claim is therefore distance-free extraction with respect to pairwise metric distances, not computation-free retrieval. In particular, DARNE can be seen as an adaptive rank-space neighbor-generation and geometric filtering framework instead of an exact metric nearest-neighbor search. Since each feature m is mapped to a discrete

ordinal lattice {1, ..., α_m}, a rank-space axis-aligned hyper-window specifies an $L_{infinity}$ ordinal neighborhood. DARNE can therefore be used in two different modes:

Algorithm 1: Rank Matrix Construction

Input: Training dataset $X \in \mathbb{R}^{(N,M)}$
Output: R, U_m, α_m
for $m = 1$ *to* M do
  $U_m = sorted_{unique}\ (\{x_{(1,m)}, x_{(2,m)}, \ldots, x_{(N,m)}\})$
  $\alpha_m = |U_m|$
  for $i = 1$ *to* N do
    $r_{(i,m)} = 1 + |\{u\ in\ U_m: u < x_{(i,m)}\}|$, $r_{(i,m)} \in \{1, \ldots, \alpha_m\}$
  end for
end for
$R = [r_{(i,m)}] \in Z^{NxM}$
return R, U_m, α_m

1. Direct Rank-Neighborhood Mode where the extracted rank-space neighbor set $W_q(Diff_{best})$ directly serves as neighborhood set for non-parametric classification or hashing without performing any metric distance computation.
2. Two-Stage Neighbor-Generation Mode where the extracted rank-space neighbor set $W_q(Diff_{best})$ is a fast ordinal pre-filter that reduces N database points to a compact neighbor set $|W_q(Diff)| \ll N$ over which exact Euclidean distances can optionally be computed if strict metric ordering is desired.

Algorithm 2: Density-Adaptive Rank space Nearest Neighbor Extraction

Input: x_q, R, U_m, α_m, $Diff_{min}$, $Diff_{max}$, β, k_{target}, k_{min}, T_{max}, ε γ, L_{max}
Output: $NN(q) = W_q$
for $m = 1$ *to* M do
  $r_{(q,m)} = \min(\alpha_m, max\ (1, 1 + |\{u\ in\ U_m: u < x_{(q,m)}\}|))$
$Diff < Diff_{min}$; $t < 0$
$A = \max_m(\alpha_m - 1)$
while $t < T_{max}$ *do*
  $Diff_m = \begin{cases} 0, & \alpha_m = 1, \\ min\left(\alpha_m, -1max\left(1, \left\lceil Diff^{\frac{\alpha_m - 1}{A}} \right\rceil\right)\right), & \alpha_m > 1. \end{cases}$
  $W_q(Diff) = \{\ i \in \{1, \ldots, N\}, i \neq q : |r_{(i,m)} - r_{(q,m)}| <= Diff_m$
  $V_q(Diff) = \prod_{m=1}^{M}\left(U_{(q,m)} - L_{(q,m)} + 1\right)$
  $\log(\lambda_q) = \log(\rho_q) - \log(\rho_{global})$
  If W_q *is empty*
    $\log(\lambda_q) = -\infty$
  else
    $J(Diff) = min\left(|, log\left(\lambda_q\right) \mid L_{max}\right) + \gamma |\ log\left(\frac{|W_q| + 0.5}{k_{\text{target}} + 0.5}\right) |$
  end
  If $|W_q| < k_{min}$
    $NN_{best} = W_q(Diff)$
  end
  If $|W_q| < k_{min}$ *or* $(\lambda_q < 1 - \varepsilon \wedge |W_q| < k_{target})$
    $Diff_{new} = min(D, iff_{max} max(D, iff + 1\lceil Diff\ (1 + \beta)\rceil))$
  if else $\lambda_q > 1 + \varepsilon \wedge |W_q| > k_{target} \wedge Diff > Diff_{min}$
    $Diff_{new} = max\left(D, iff_{min} min\left(D, iff - 1\left\lfloor \frac{Diff}{1+\beta} \right\rfloor\right)\right)$
  end
  $|W_q(Diff)| \geq k_{min}$
  $Diff_{best} = argmin\ (J(Diff)$
  $Diff = Diff_{new}$, $t = t + 1$
return $\mathbf{NN(q) = W_q(Diff_{best})}$

3.1. Data Model and Notation

For classification tasks, let

$$D = \{(x_i, y_i)\}_{i=1}^{N}, \quad x_i \in \mathbb{R}^M, y_i \in \{1, \ldots, C\} \tag{1}$$

where N is the number of samples, M is the number of features, and y_i is optional because DARNE can be used for supervised classification, unsupervised retrieval, hashing, or image-patch processing. The query point is denoted x_q. If the query is itself a training point, its own index is excluded from the returned neighbor set unless the task explicitly requires self-neighbor inclusion. For reference, the naive distance-based retrieval cost that DARNE avoids is

$$dist(q, x_i) = \sqrt{\sum_{m=1}^{M} \left(x_{q,m} - x_{i,m}\right)^2} \text{ for } i = 1, \ldots, N \tag{2}$$

which requires $O(NM)$ query-time arithmetic before neighbor ordering or selection of a dominant cost in image patch tasks where the operation may be repeated for millions of pixels or patches.

3.2. Rank-Space Transformation

For each feature dimension m, the unique observed values are sorted, and dense ranks are assigned so that identical feature values receive the same rank:

$$U_m = sorted_{unique}\left(\{x_{(1,m)}, x_{(2,m)}, \ldots, x_{(N,m)}\}\right) \tag{3}$$

$$\alpha_m = |U_m| \tag{4}$$

$$r_{(i,m)} = 1 + |\{u \text{ in } U_m : u < x_{(i,m)}\}|, \; r_{(i,m)} \in \{1, \ldots, \alpha_m\} \tag{5}$$

$$R = [r_{(i,m)}] \in Z^{NxM} \tag{6}$$

Here α_m is the number of unique values in feature m. Dense ranking provides a stable ordinal representation and prevents duplicated values from creating arbitrary rank gaps. Normalized ranks $\check{r}_{i,m} = r_{(i,m)}/\alpha_m$ are reserved for a separate ablation; the main formulation uses raw integer dense ranks to avoid mixing integer *Diff* values with normalized $[0, 1]$ ranks. Table 2 presents the synthetic dataset, which is transformed into rank- space representation as shown in Table 2 with the help of Equations (3)–(6).

Table 2. Synthetic dataset comprising continuous attribute values across multiple features.

	D1	D2	D3	D4	D5	D6	D7	D8	D9	D10
Feature 1	0.9	2.8	1.1	1.1	2.2	3.35	4.5	4.5	6.0	3.0
Feature 2	1.35	1.35	1.60	1.05	1.90	1.35	1.99	1.05	1.35	1.99

The rank-space transformation (Equations (3)–(6)) has an important property in theory: the number of unique empirical training values between $x_{(i,m)}$ and $x_{(q,m)}$ along feature m-approximating $N_m|\ |U_m(x_{(i,m)}) - U_m(x_{(q,m)})|$ is used to approximate$|\ |x_{(i,m)} - x_{(q,m)}|$. Therefore, the distance in rank space is not the same as the distance in the Euclidean space. Here are two concrete phenomena, both directly visible in the synthetic benchmark of Tables 2 and 3: (i) Small Rank Distance with Large Euclidean Distance (Marginal Gap Compression): Take a look at query point D6 = (3.35, 1.35) and its rank coordinates r6 = (6, 2). Its true Euclidean nearest neighbor in Table 2 is D2 = (2.80, 1.35) at Euclidean distance d_2(D6, D2) = 0.5500, followed by D10 = (3.00, 1.99) at d_2(D6, D10) = 0.7295. However, because D10 has Feature 1 value 3.00 (rank 5) lying between D2 (2.80, rank 4) and D6 (3.35, rank 6), the $L_{infinity}$ rank distance between D6 and D2 is max $(|6 - 4|, |2 - 2|) = 2$. Meanwhile, point D8 = (4.50,

1.05) with rank r8 = (7, 1) has $L_{infinity}$ rank distance max (|7 − 6|, |1 − 2|) = 1 from D6, even though its Euclidean distance is d_2(D6, D8) = 1.1885 (more than twice the Euclidean distance of D2). Therefore, a window W_D6($Diff$ = 1) will select {D8} without taking the "truly" Euclidean nearest neighbor of D6, D2. (ii) Large Rank Distance with Small Euclidean Distance (Dense Marginal Inflation): Conversely, consider D7 = (4.50, 1.99) with rank r7 = (7, 5) and D8 = (4.50, 1.05) with rank r8 = (7, 1). In Euclidean space, D7 and D8 are mutually each other's closest neighbor at d_2(D7, D8) = 0.9400. Yet in Feature 2, multiple points take intermediate values {1.35, 1.60, 1.90} (ranks 2, 3, 4), inflating their rank-space $L_{infinity}$ distance to max (|7 − 7|, |5 − 1|) = 4, whereas D8 has rank distance 1 to D9 = (6.00, 1.35) despite a much larger Euclidean distance d_2(D8, D9) = 1.5297. This analytical proof shows that DARNE retrieves ordinal 'rank-neighbors', not guaranteed metric nearest neighbors, which justifies the Exact-k-NN Recall metric in Section 6.1.

Table 3. Rank-ordered representation of the synthetic dataset following the proposed transformation.

	D1	D2	D3	D4	D5	D6	D7	D8	D9	D10
Feature 1	1	4	2	2	3	6	7	7	8	5
Feature 2	2	2	3	1	4	2	5	1	2	5

3.3. Query Processing for Unseen Data Points

For an unseen query x_q, its rank coordinate in feature m is obtained by counting the number of training values smaller than $x_{(q,m)}$, clipped to the valid rank interval:

$$r_{(q,m)} = \min(\alpha_m, max\ (1, 1 + |\{u\ in\ U_m\colon u < x_{(q,m)}\}|)) \quad (7)$$

This keeps the query on the same rank lattice as the training data and avoids undefined behavior for values outside the training feature range.

3.4. Boundary-Corrected Rank-Space Hyper-Window

For a scalar window half-size $Diff$, or a per-dimension vector $Diff = (Diff_1,\ldots, Diff_M))$, DARNE defines the query-centered hyper-window in rank space as

$$W_q(Diff) = \{\, i \in \{1, \ldots,N\}, i \neq q\colon |r_{(i,m)} - r_{(q,m)}| \leq Diff_m\ \forall m = 1,\ldots,M\} \quad (8)$$

A naive lattice-volume expression such as $(2.Diff + 1)^M$ is insufficient because the window may be clipped near rank boundaries and may use feature-specific widths. The boundary corrected volume uses

$$L_{(q,m)}(Diff_m) = \max(1, r_{(q,m)} - Diff_m) \quad (9)$$

$$U_{(q,m)}(Diff_m) = \min(\alpha_m, r_{(q,m)} + Diff_m) \quad (10)$$

$$V_q(Diff) = \prod_{m=1}^{M} \left(U_{(q,m)} - L_{(q,m)} + 1\right) \quad (11)$$

3.5. Local and Global Rank-Space Density Estimation

DARNE treats the rank lattice as an occupancy structure. The global rank-space volume and density are

$$V_{global} = \prod_{m=1}^{M} \alpha_m \quad (12)$$

$$\rho_{global} = N / V_{global} \quad (13)$$

and the local density around query q at window $Diff$, and its ratio to the global density, is:

$$\rho_d(Diff) = |W_q(Diff)| / V_q(Diff) \quad (14)$$

$$\lambda_q(Diff) = \rho_d(Diff)/\rho_{global} \tag{15}$$

If λ is considerably larger than 1, the local region is denser than the global average; if λ is much less than 1, the local region is sparse. Adaptive window selection is driven by this ratio. DARNE is therefore best described as a low-tuning, self-adaptive framework rather than a parameter-free method. Control settings such as the stability threshold, maximum iterations, and clipping bounds are examined through sensitivity analysis in Section 5.

3.6. Adaptive Hyper-Window Selection

The control law updates $Diff$ using a density-ratio-guided rule with integer rounding and monotonic safeguards. Let $k_{\min}$ denote the minimum acceptable neighbor count and k_{target} the preferred neighbor count. Expansion is triggered when the neighbor set is empty, $|W_q| < k_{\min}$, or density is low while the neighbor count remains below k_{target}; contraction is triggered only when density is high, $|W_q| > k_{\text{target}}$ and $Diff > Diff_{\min}$:

$$Diff_{new} = \min(Diff_{max}, max(Diff_{old} + 1, \lceil Diff_{old}(1+\beta) \rceil)) \tag{16}$$

for expansion:

$$Diff_{new} = \max\left(Diff_{min}, min\left(Diff_{old} - 1, \left\lfloor \frac{Diff_{old}}{1+\beta} \right\rfloor\right)\right) \tag{17}$$

for contraction:

$$1 \le Diff_{min} \le Diff \le Diff_{max} \le A \text{ , where } A = max_m(\alpha_m - 1), \text{for } A \ge 1 \tag{18}$$

where β is a smoothing coefficient. Because rank-space occupancy changes discretely, $\lambda_q(Diff)$ need not vary smoothly; a bounded-search fallback additionally requires $|W_q(Diff)| \ge k_{\min}$ rather than a merely non-empty set, allowing an ablation to isolate whether performance gains come from the density-ratio update or the density objective itself. Figure 1 illustrates the underlying motivation: with a fixed window ($Diff = 1$) applied to a ten-point synthetic dataset, points D1 and D8 correctly find neighbors, but D2, D7, and D10 are left without any neighbor inside the fixed window, motivating a window size that adapts to local density. With fixed-window FGENN [17] a window expands $Diff$ iterations at a time (*100 iterations on SONAR, 30 iterations on Covertype*) and neighbors are extracted at every set of fixed parameters.

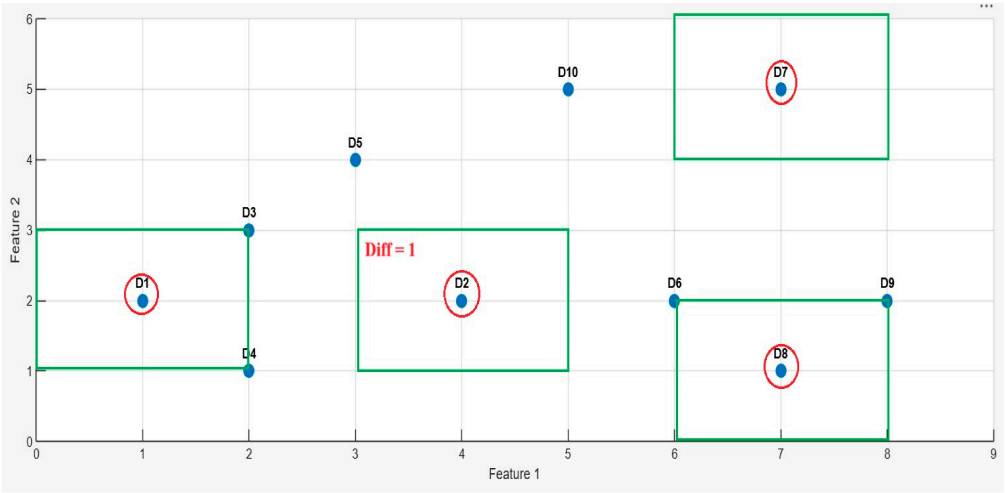


Figure 1. Schematic illustration of the limitation associated with a fixed rank-space window when ($Diff = 1$): several query points (D2, D7, D10) fall outside the window of any neighbor, motivating density-adaptive window selection.

3.7. Neighbor-Set Retrieval Strategies

One implementation is developed and compared. The naive implementation scans all N samples and checks the M window conditions for each query i,

$$\left| r_{(i,m)} - r_{(q,m)} \right| \leq \ Diff_m, \forall m = 1, \ldots, M \tag{19}$$

3.8. DARNE Algorithms

The preprocessing stage (rank-matrix construction) is summarized in Algorithm 1; the adaptive query time extraction stage is summarized in Algorithm 2. The corresponding flowcharts are shown in Figures 2 and 3. The proposed method is implemented by using Equations (3)–(18). The notation used in algorithm is defined as follows: X is the training dataset, N is the total available data in X, M is the number of features. U_m is a sorted set of different training values in feature m. α_m is the number of unique training values of feature m. $R_{(i,m)}$ is the rank without considering duplicate values for the (i,m)-th data element. where x_q is the query vector. The training-row identifier IDx_q is only given if the query coincides with the record of a training-row and it is only this row that is excluded. Equal samples with different row identifiers will still be eligible. *Diff* is the scalar window control for a feature, defined by its half-width $Diff_m$, $A = max_m(\alpha_m - 1)$, and $n\ (Diff) = |W_q(Diff)|$. U_m signifies the training values of the feature. $L_{(q,m)}$ and $U_{(q,m)}$ are the lower and upper bounds of the window.

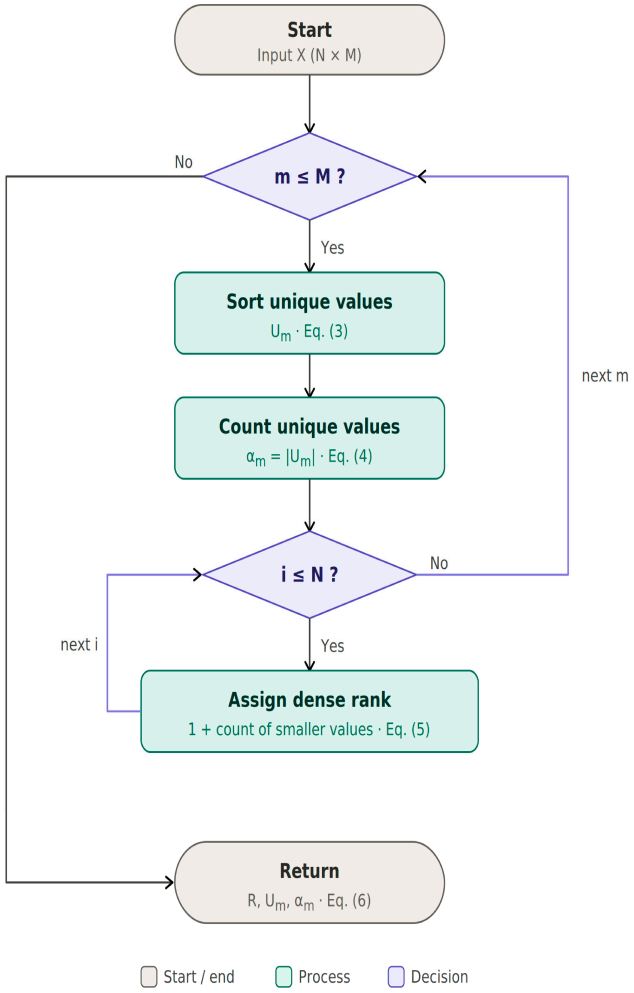


Figure 2. Flowchart of Algorithm 1 for rank-matrix construction. Equation numbers indicate where each step is defined.

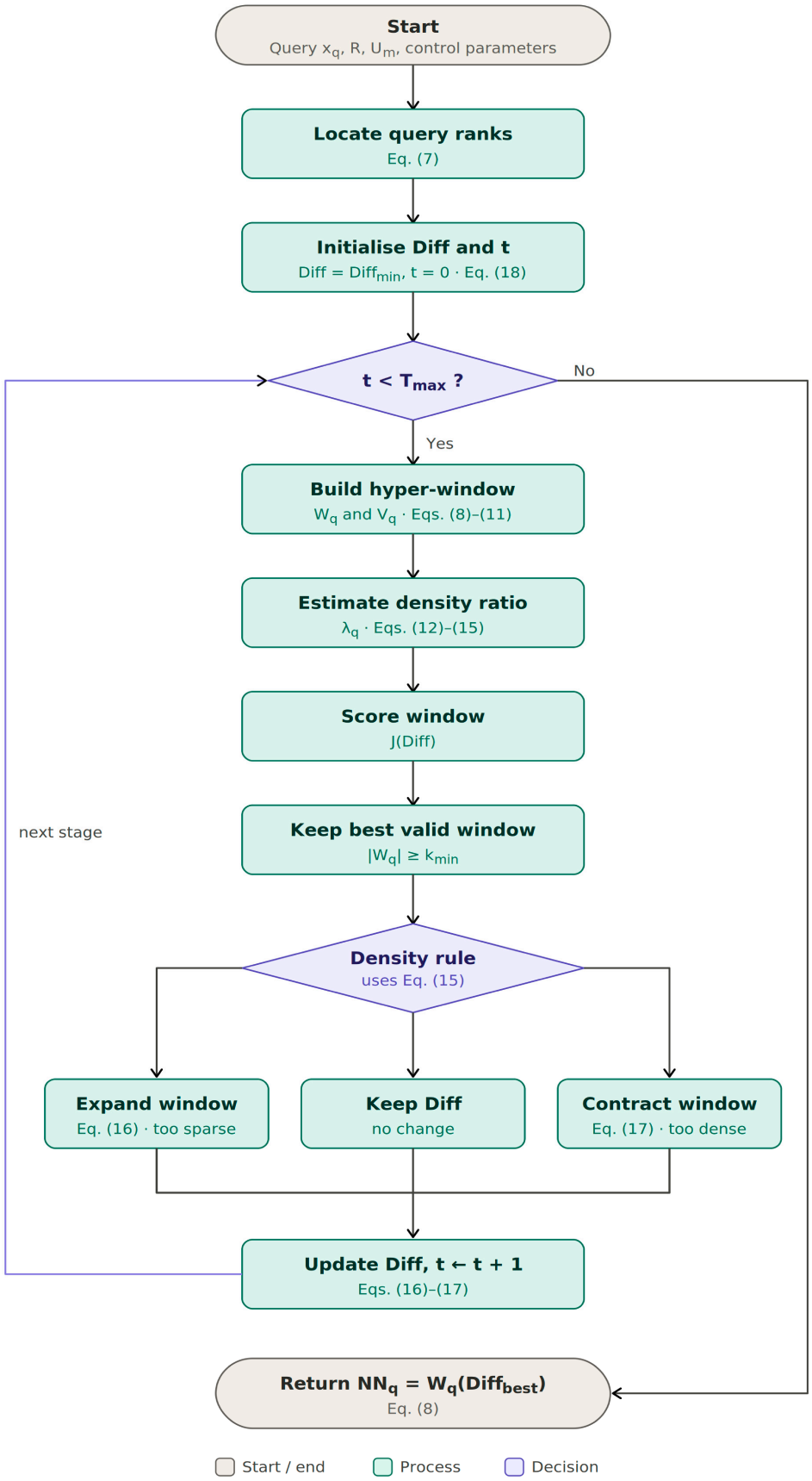


Figure 3. Flowchart of Algorithm 2 for density-adaptive rank-space neighbor extraction. Equation numbers indicate where each step is defined.

Control parameters $Diff_{min}$ and $Diff_{max}$ are integer scalar bounds and must satisfy Equation (18) when A ≥ 1; β controls the magnitude of the multiplicative update; $\varepsilon \leq 1$ is the upper bound of the ratio difference density; k_{min} and k_{target} are the positive integer neighbor count settings; T_{max} is the positive integer that limits the number of evaluated windows; $\gamma \geq 0$ counts deviation as a weight and L_{max} (>0) caps the density-deviation item. The valid setting of the neighbor count should satisfy $k_{min} \leq k_{target} \leq$ number of eligible training records. The ceiling and floor operations are used for rounding up and down, respectively, and log is the natural logarithm.

The direct mode computes at most the initial window, when the initial window is a single updated window. Therefore, in this mode the maximum number of evaluation windows is set to 2. In the bounded iterative mode, apart from the same procedure, the number of iterations is limited to a prescribed T_{max}; this stage is represented by $Ittr_{max}$. The comparison of density determines the decision of expansion or contraction. β is the update step size in Equations (16) and (17).

3.9. Classification Decision Rule

For supervised classification, DARNE returns a variable-size neighbor set; class prediction uses majority vote or rank window occupancy weighting:

$$P(C|x_q) = |\{\, i \in NN_q : y_i = c\}|/|NN_q| \tag{20}$$

$$\hat{y}_q = argmax_c P\left(C\middle|x_q\right) \tag{21}$$

If NN(q) remains empty after the maximum number of search iterations, DARNE applies a documented fallback nearest available rank-interval expansion up to $Diff_{max}$ or a conventional distance-based fallback with empty-set handling reported explicitly rather than hidden.

4. Structural Properties and Computational Complexity

In addition to the empirical comparisons in terms of runtime and accuracy, we provide four formal structural and complexity guarantees for DARNE that are summarized in Table 4 and proven in Propositions 1–4 shown below:

1. Component-wise monotonicity of the rank-space hyper-window.
2. Exactness of the structure under monotone per-feature transformations.
3. No oscillation and guaranteed finite termination.
4. Exact time and space complexity in offline preprocessing, naive query scanning and indexed rank-interval retrieval.

Proposition 1 (Component-Wise Monotonicity). *For any non-negative window vectors* Diff ≤ Diff′ *(component-wise) and query* x_q *with rank coordinates* r_q*, the following is satisfied:*

$$W_q(\text{Diff}) \subseteq W_q(\text{Diff}') \text{ and } 1 \leq V_q(\text{Diff}) \leq V_q(\text{Diff}') \leq V_{\text{global}}.$$

Proof.

- By Equation (8), $i \in W_q(Diff) \Longleftrightarrow |r_{i,m} - r_{q,m}| \leq Diff_m$ for all $m \in \{1,\ldots,M\}$. Since $Diff_m \leq Diff'_m$, transitivity implies $|r_{i,m} - r_{q,m}| \leq Diff'_m$, yielding $W_q(\text{Diff}) \subseteq W_q(\text{Diff}')$.
- By Equations (9) and (10), $L_{q,m}(Diff'_m) \leq L_{q,m}(Diff_m)$ and $U_{q,m}(Diff'_m) \geq U_{q,m}(Diff_m)$.
- Because each 1D interval length $(U_{q,m} - L_{q,m} + 1) \geq 1$ is non-decreasing in $Diff_m$, taking the product over M dimensions via Equations (11) and (12) gives

$$1 \leq V_q(\text{Diff}) \leq V_q(\text{Diff}') \leq \prod_{m=1}^{M} \alpha_m = V_{\text{global}}.$$

□

Proposition 2 (Monotone Transformation Invariance). *Let* $\{g_m : \mathbb{R} \to \mathbb{R}\}_{m=1}^{M}$ *be strictly increasing functions. Under coordinate-wise mappings* $\tilde{x}_{i,m} = g_m(x_{i,m})$ *and* $\tilde{x}_{q,m} = g_m(x_{q,m})$, *the rank matrix* R, *cardinalities* $\{\alpha_m\}$, *query ranks* r_q, *volumes* $V_q(Diff)$, *density ratios* $\lambda_q(Diff)$, *and neighbor set* $\mathrm{NN}(q)$ *remain strictly invariant.*

Proof.

Strictly increasing functions preserve pairwise order and equality:

- $x_{i,m} < x_{j,m} \iff g_m(x_{i,m}) < g_m(x_{j,m})$ and $x_{i,m} = x_{j,m} \iff g_m(x_{i,m}) = g_m(x_{j,m})$. Thus, the sorted unique set $\widetilde{U}_m$ (Equation (3)) preserves $\tilde{\alpha}_m = \alpha_m$ (Equation (4))
- Identical strict-predecessor counts in Equations (5) and (7), yielding $\widetilde{R} = R$ and $\tilde{r}_q = r_q$.
- Because Equations (8)–(17) depend solely on R, r_q, and $\{\alpha_m\}$, all subsequent quantities $W_q(Diff)$, $V_q(Diff)$, $\lambda_q(Diff)$, and $\mathrm{NN}(q)$ are invariant.

□

Proposition 3 (Finite Termination and Non-Oscillation). *For any dataset* $X \in \mathbb{R}^{N \times M}$ *and query* $x_q \in \mathbb{R}^M$, *Algorithm 2 terminates in* $T \le T_{\max}$ *stages* ($T = 2$ *in direct mode*) *without oscillation.*

Proof.

- Algorithm 2 restricts $Diff$ to the finite integer lattice $[Diff_{\min}, Diff_{\max}] \cap \square$, with $Diff_{\min} = 1$ and $Diff_{\max} \le \max_m(\alpha_m - 1) \le N - 1$ (Equation (18); Line 4).
- In direct two-stage execution, stage $t = 1$ evaluates the initial density ratio λ_q (Equation (15)) and stage $t = 2$ applies the updated window (Equations (16) and (17)), terminating when $|W_q(Diff)| \ge k_{\min}$.
- If $|W_q(Diff)| < k_{\min}$, only the expansion rule.
- $Diff_{\text{new}} = \min(Diff_{\max}, \max(Diff + 1, \mathrm{round}(Diff(1 + \beta))))$ (Lines 18–19) is active.
- Since $Diff_{\text{new}} \ge Diff + 1$ for $Diff < Diff_{\max}$, $Diff$ increases strictly until $|W_q| \ge k_{\min}$ or $Diff = Diff_{\max}$ in at most $\min\left(T_{\max}, \max_m \alpha_m - 1\right)$ steps.
- Selecting $Diff_{\text{best}} = \mathrm{argmin} J(Diff)$ over the finite visited set (Line 24) guarantees termination.

□

Proposition 4 (Asymptotic Complexity). *For* N *training samples and* M *features: (i) Offline preprocessing (Algorithm 1) takes* $O(MN\log N)$ *time and* $O(NM)$ *space; (ii) Online extraction (Algorithm 2) takes* $O(NM)$ *time under naive scanning or* $O\left(M\log N + 2M\overline{K}\right)$ *time in indexed retrieval, where* $\overline{K}$ *is the mean 1D neighbor size.*

Proof.

- Offline: Sorting and deduplicating each feature (Equation (3)) take $O(N\log N)$ time, binary-search rank assignment (Equation (5)) costs $O(N\log\alpha_m) \le O(N\log N)$ per feature, totaling $O(MN\log N)$ across M features. Storing $R \in \square^{N \times M}$, $\{U_m\}_{m=1}^{M}$ ($\sum_m \alpha_m \le NM$), and inverted rank buckets $\{I_m[r]\}$ requires $O(NM)$ space.
- Online: Binary search for r_q (Equation (7)) takes $O\left(\sum_{m=1}^{M} \log \alpha_m\right) \le O(M\log N)$ time, and evaluating V_q and λ_q (Equations (9)–(15)) takes $O(M)$ operations per stage. Naive scanning (Equation (19)) tests N points across M features in $O(NM)$ time per stage, yielding $O(M\log N + TNM) = O(NM)$ for $T = 2$. Indexed retrieval traverses active bucket slices $[L_{q,m}, U_{q,m}]$ with mean cardinality $\overline{K} = \frac{1}{M}\sum_{m=1}^{M}\sum_{r=L_{q,m}}^{U_{q,m}} |I_m[r]|$ in $O\left(M\overline{K}\right)$ time per stage, totaling $O\left(M\log N + 2M\overline{K}\right)$ time (plus $O(|\mathrm{NN}(q)|)$ for majority voting in Equations (20) and (21)).

□

https://doi.org/10.3390/a19110949

Known risks are treated as explicit failure modes rather than hidden limitations. Rank-space hypercubes may include geometrically inappropriate points when features are strongly correlated or class manifolds are curved; very high dimensionality can make rank-space occupancy extremely sparse, destabilizing the density ratio; dense ranks may lose discriminative information under heavy value tying, particularly for low-cardinality categorical features; local density adaptation improves neighbor coverage but does not by itself guarantee optimal classification boundaries; and indexing reduces query time only when interval neighbor sets remain small, since dense windows can still incur high neighbor-intersection cost.

Table 4. Formal structural properties, mathematical guarantees, and asymptotic complexity bound of DARNE.

Property	Formal Guarantee/Bound	Operational Significance
Monotonicity	$Diff \leq Diff' \Longrightarrow W_q(Diff) \subseteq W_q(Diff')$, $V_q(Diff) \leq V_q(Diff')$ (**Prop. 1**)	Ensures monotonic neighbor expansion and non-decreasing lattice volume.
Scale Invariance	For strictly increasing $\{g_m\}$, R, V_q, λ_q, and $\mathrm{NN}(q)$ are invariant (**Prop. 2**)	Eliminates sensitivity to feature units, linear rescaling, and monotone skew.
Finite Termination	Terminates in $T = 2$ stages (direct) or $T \leq \min\left(T_{\max}, \max_m \alpha_m - 1\right)$ (**Prop. 3**)	Prevents oscillation and bounds worst-case query latency.
Boundary Correction	$V_q(Diff) = \prod_{m=1}^{M}\left(U_{q,m} - L_{q,m} + 1\right) \in \left[1, V_{\text{global}}\right]$	Prevents artificial density deflation near feature boundaries.
Tie Handling	Identical values map to identical dense ranks $r_{i,m} \in \{1,\ldots,\alpha_m\}$, $\alpha_m \leq N$	Avoids arbitrary rank gaps in discretized or tied attributes.
Complexity	Offline: $O(MN\log N)$ time, $O(NM)$ space; Online: $O(NM)$ [Scan] or $O\left(M\log N + 2M\overline{K}\right)$ [Indexed] (**Prop. 4**)	Decouples offline rank-sorting from fast two-stage online extraction.

5. Experimental Methodology

The experimental protocol is designed to test the research hypotheses across small interpretable datasets, medium and large tabular datasets, high-dimensional datasets, synthetic density-control datasets, image-patch datasets, and retrieval/hashing tasks.

5.1. Datasets

Table 5 summarizes the dataset groups. Small interpretable tabular sets (IRIS, Glass, and Hayes-Roth [55]) allow direct visualization of neighbor sets; medium tabular sets (SONAR, Abalone, Breast Cancer [55]) evaluate accuracy, query time, and neighbor stability; large tabular sets (≥200k samples where available) measure scalability and index behavior; synthetic density-gradient datasets (Gaussian mixtures, anisotropic clusters, overlapping bands, and controlled outliers) test dense/sparse adaptation under known ground truth; high-dimensional data (text vectors, sparse features, and embeddings) evaluate hubness and neighbor explosion; and hashing/retrieval datasets are evaluated with P@r, P@TopN, and mAP.

Table 5. Dataset groups included in the experimental evaluation.

Dataset Group	Examples	Purpose
Small interpretable	IRIS, Glass, Hayes-Roth	Visualize neighbor sets; baseline comparison
Medium tabular	SONAR, Abalone, Breast Cancer	Accuracy, query time, neighbor stability
Large tabular	≥200k-sample sets	Scalability and index behavior
Synthetic density	Gaussian mixtures, anisotropic clusters	Dense/sparse and boundary adaptation
High-dimensional	Text vectors, sparse embeddings	Hubness, sparsity, neighbor explosion
Hashing/retrieval	P@r, P@TopN, mAP-compatible sets	Hashing/retrieval evaluation axis

5.2. Comparison Methods

DARNE is compared against classical k-NN with cross-validated k, weighted k-NN with distance-weighted voting, adaptive k-NN with locally estimated k, fuzzy and evidential k-NN, FGENN with multiple fixed *Diff* values (including the best validated fixed-*Diff* setting), and locality-sensitive hashing for large-scale retrieval comparisons.

All baseline methods and DARNE adhere to a standard hyperparameter configuration to enable a fair comparison of algorithms and their hyperparameters.

1. Traditional k-NN and FGENN are presented at a fixed reference $k = 3$ and preset dataset windows (*Diff* = 2,3 on IRIS; *Diff* = 100 on SONAR; *Diff* = 30 on Covertype) only to view raw neighbor sets and number of iterations.
2. Each baseline is set using hyperparameter validation on the training partitions: Classical k-NN tries different values of k in {1, 3, 5, 7, 9, 11} using inner cross-validation of the training partitions; Weighted k-NN weights votes using inverse squared Euclidean distance.

$$w_i = \frac{1}{d\left(x_q, x_i\right)^2 + \epsilon} \tag{22}$$

Fuzzy k-NN is based on per-query fuzzy exponent $m = 2.0$; adaptive k-NN is based on adapting $k_q \in [k_{min}, k_{max}] = [3, 15]$ based on local ball density; FGENN is based on choosing the best fixed *Diff* from the set of predefined Diffs on the dataset grid; and DARNE is based on fixed default control parameters ($k_{min} = 3$, $k_{target} = 5$, $\epsilon = 0.10$, and $T = 2$ stages) without manual retuning per query. Additionally, we report analytical and empirical comparison with space-partitioning indexing baselines (KD-Tree [24] and Ball-Tree [25]) to provide a context for query-time speedups for low- and high-dimensional data.

In Section 2, we briefly overview a variety of recent k-NN extensions [35–42]; the five experimental baselines analyzed in Section 6 were carefully chosen to create a mathematically sound and orthogonal experimental factorial control over the three essential design dimensions of nearest-neighbor learning:

1. Representation Space: the four metric-distance k-NN classifiers were compared with the two rank space classifiers (FGENN and DARNE) and isolated;
2. Neighborhood Size Control (Fixed Global Parameter vs. Local Density-Adaptive Parameter): The fixed-k Classical k-NN as compared to query-adaptive k_q in Adaptive k-NN [12,36] is isolated in metric space, and the fixed-*Diff* FGENN [18] as compared to the density-adaptive DARNE is isolated in rank space.
3. Decision Aggregation Rule: Classical k-NN versus Weighted k-NN [19] and Fuzzy k-NN [20] are compared and isolated.

Notably, our procedural Adaptive k-NN baseline does include the essential ingredient of the state-of-the-art in locally adaptive *k*-selection; that is, the dynamic selection of a query dependent neighborhood size in $[k_{\min}, k_{\max}]$ according to the neighborhood density around x_q measured by the distance-ball. In addition to this, based on Section 2.2, the recent metric classifiers, such as weight-updated iterative k-NN [37], multi-distance kernel fusion [38] and distributed evidential k-NN [41,42], all need to compute the same metric pairwise distance vector among x_q and all N training samples prior to using their respective iterative reweighting or Dempster–Shafer fusion rules. Thus, their query time-latency is lower-bounded by Classical, Weighted, Adaptive and Fuzzy k-NN, and in addition their continuous boundary-smoothing characteristic is directly represented by Weighted, Adaptive and Fuzzy k-NN. IRIS, SONAR and Covertype have a complete hyperparameter and tuning sensitivity parameter for experimental reproducibility described in Table 6. Control parameter clipping bounds ($Diff_{\min}$, $Diff_{\max}$), minimum and target neighbor counts ($k_{\min}$, k_{target}) and iteration bounds ($Ittr_{max}$). Sensitivity parameters: beta(β), an expansion/contraction smoothing factor; epsilon(ε), a density-ratio stability tolerance; and gamma(γ), a count-deviation penalty weight.

Table 6. Hyperparameter configurations and sensitivity analyses for the IRIS, SONAR, and Covertype datasets.

Parameter	IRIS	SONAR	Covertype
$k_{min} - k_{\text{target}}$	3–5	3–10	3–10
$Diff_{min} - Diff_{max}$	1–5	85–120	20–50
β	0.50	0.70	0.50
ε	0.10	0.10	0.10
γ	0.50	0.50	0.50
$Ittr_{max}$	3	2	3

5.3. Evaluation Methodology

Table 7 lists the metrics used per task. Classification is evaluated with accuracy, macro-F1, balanced accuracy, per-class recall, and boundary-region accuracy; neighbor extraction is evaluated by neighbor count, class purity, overlap with exact k-NN, empty-neighborhood rate, and dense/sparse-region behavior; runtime is evaluated by preprocessing time, query latency, throughput, scaling with N and M, and memory usage; and retrieval/hashing is evaluated with precision at radius, P@TopN, mean average precision, and retrieval latency.

Table 7. Evaluation metrics used for each experimental task.

Task	Metrics
Neighbor extraction	Neighbor count, class purity, overlap w/exact k-NN, empty-set rate
Runtime	Preprocessing time, query latency, throughput, scaling, memory
Classification	Accuracy, macro-F1, balanced accuracy, per-class recall, boundary accuracy
Retrieval/hashing	P@r, P@TopN, mAP, retrieval latency

5.4. Validation and Experimental Protocol

Repeated stratified *k*-fold cross-validation is used for classification where labels are available; fixed train/test splits with repeated seeds are used for large datasets where repeated cross-validation is impractical. Preprocessing costs are reported separately from query-time costs; baseline hyperparameters are tuned only on validation folds and never on test data; hardware, software versions, random seeds, and repetition counts are reported;

and paired statistical tests with confidence intervals are used for method comparisons rather than single-run scores alone.

All experiments are conducted on HP Z4 Workstation (HP Inc., Palo Alto, CA, USA), which are 64-bit x86_64 architecture, an Intel(R) Core(TM) i7 processor, 2.80 GHz base clock frequency, 4 physical cores/8 logical threads, 32 GB RAM, and running 64-bit Microsoft Windows 11 Pro and MATLAB R2023b (Version 23.2). The high-resolution wall-clock times are collected using tic/toc and timeit with $n = 10$ independent runs on IRIS, SONAR and Covertype.

6. Experimental Results

Implementation and testing were carried out in MATLAB on the IRIS, SONAR and Large Dataset (Covertype) to validate the DARNE formulation prior to the larger multi-domain study described in Section 5. Results are discussed with respect to the research hypotheses on nearest neighbor extraction, computational efficiency, classification accuracy, boundary robustness, and neighbor-set behavior. All experiments were conducted on a standard Intel Core i7 (2.80 GHz, 32 G RAM, 64-bit Windows) workstation (described in Section 5.4), with the same hyperparameter settings provided in Table 6.

6.1. Nearest-Neighbors Extraction Performance

Tables 8–10 report the raw neighbor-extraction behavior of Traditional k-NN, FGENN, and DARNE on five representative query indices from the IRIS, SONAR, and large-scale Covertype datasets, respectively, across a range of *Diff* (window) settings. Traditional k-NN returns its neighbor set directly from a fixed k ($k = 3$ in every dataset) via exhaustive pairwise distance ranking, with no window or iteration parameter involved. FGENN instead fixes a single *Diff* value per dataset *Diff* = 2, 3 for IRIS, *Diff* = 100 for SONAR, and *Diff* = 30 for Covertype and its number of search iterations is tied directly to that fixed *Diff*, so the geometric hyper-window must be swept through as many iterations as the preset *Diff* before the neighbor set is finalized (e.g., 100 iterations per query on SONAR and 30 on Covertype), regardless of how sparse or dense the local neighborhood.

Table 8. Neighbor-extraction performance of the evaluated methods on the IRIS dataset.

Idx	Traditional k-NN		FGENN			DARNE (Proposed)		
	k	Neighboring Indexes	*Diff*	Iter.	Neighboring Indexes	Adaptive *Diff*	Iter.	Neighboring Indexes
11	3	28, 37, 49	2	2	6, 17, 28, 37, 49	2	2	6, 17, 28, 37, 49
34	3	16, 17, 33	2	2	16	3	2	6, 15, 16, 17, 33
70	3	81, 82, 90	2	2	54, 68, 81, 82, 83, 90, 93	2	2	54, 68, 81, 82, 83, 90, 93
99	3	58, 61, 94	2	2	58, 94	5	2	58, 61, 82, 94
119	3	106, 123, 136	3	3	123	5	2	106, 108, 123, 131, 136

Table 9. Neighbor-extraction performance of the evaluated methods on the SONAR dataset.

Idx	Traditional k-NN		FGENN			DARNE (Proposed)		
	k	Neighboring Indexes	*Diff*	Iter.	Neighboring Indexes	Adaptive *Diff*	Iter.	Neighboring Indexes
59	3	60, 61, 62	100	100	58, 60, 61, 62, 63	85	2	58, 60, 61, 62, 63
70	3	71, 72, 73	100	100	53, 55, 71, 72, 73	99	2	53, 55, 71, 72, 73
159	3	158, 160, 161	100	100	158, 160, 161	117	2	157, 158, 160, 161, 162
186	3	181, 183, 184	100	100	181, 182, 184	108	2	181, 182, 183, 184, 185
205	3	203, 204, 206	100	100	190, 195, 204, 206	102	2	55, 190, 195, 204, 206

Table 10. Neighbor-extraction performance of the evaluated methods on the Covertype dataset.

Idx	Traditional k-NN		FGENN			DARNE (Proposed)		
	k	Neighboring Indexes	*Diff*	Iter	Neighboring Indexes	Adaptive *Diff*	Iter	Neighboring Indexes
14,639	3	9970, 8210, 8158	30	30	[ ]	48	2	9970, 9992, 9998
412	3	419, 483, 397	30	30	393, 397, 419, 427, 477, 478, 483	25	2	397, 419, 427, 477, 478, 483
4245	3	4246, 4244, 4189	30	30	4189, 4190, 4244, 4246, 4305	32	2	4189, 4190, 4243, 4244, 4246, 4304, 4305
3634	3	3684, 3592, 3685	30	30	3483, 3592, 3684	34	2	3483, 3489, 3592, 3684
5820	3	5619, 5597, 6065	30	30	5619, 5965	40	2	5597, 5619, 5965

DARNE removes this dependence on a single global *Diff*. Its Adaptive *Diff* is estimated per query from the local occupancy/density structure and converges in exactly two iterations regardless of dataset size or the magnitude of the resulting window: the first iteration evaluates the local rank-space density around the query and derives a provisional *Diff* from the local-to-global density ratio, while the second iteration re-applies this corrected *Diff* to extract the final neighbor set NN(q). As Tables 8–10 show, the Adaptive *Diff* produced by this two-step process varies substantially with local density and dataset scale from 2 to 5 on IRIS, 85 to 117 on SONAR, and 25 to 47 on Covertype, yet the iterations stay fixed at two in every case, in contrast to FGENN and k-NN, whose iteration cost scales with, or is entirely determined by, the fixed window/*K* they are given. This confirms that the density-adaptive control law lets DARNE reach an appropriately sized, query-specific neighborhood without the exhaustive, *Diff*-length search that FGENN requires and without the arbitrary fixed-*k* assumption underlying Traditional k-NN.

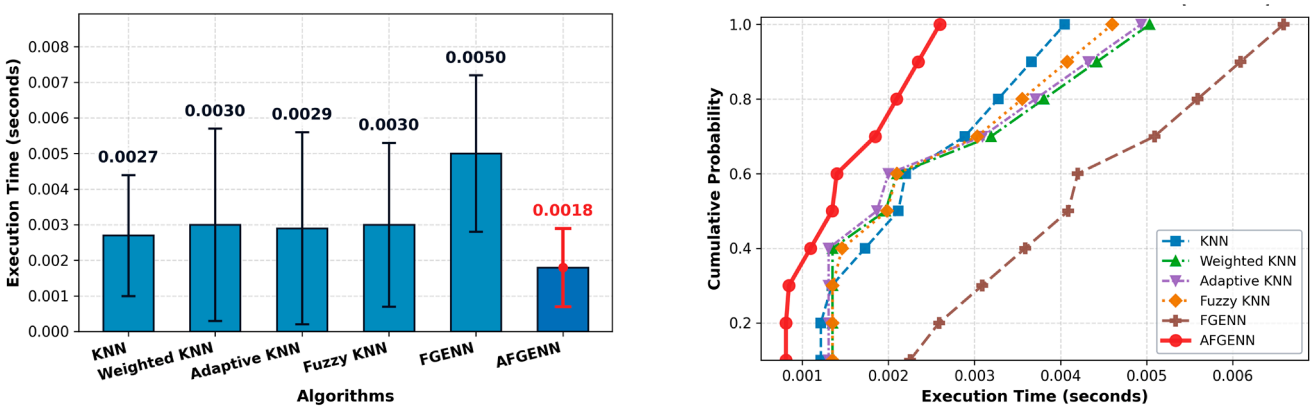
Because, instead of Euclidean distances, DARNE uses an ordinal hyper-window in rank space to extract neighbors, it is important to characterize how many NN are recovered by $W_q(Diff_{best})$ and how many of the points that are extracted are purely ordinal 'rank-neighbors.

6.2. Computational Efficiency

Table 11 and Figure 4 compare the mean query runtime, median runtime, standard deviation, and speedup of DARNE against five baselines. DARNE achieved the lowest mean runtime (0.0018 s) and the lowest variance (std. 0.0011 s) among all six methods, outperforming Classical k-NN (0.0027 s), Weighted k-NN (0.0030 s), Adaptive k-NN (0.0029 s), Fuzzy k-NN (0.0030 s), and fixed-window FGENN (0.0050 s). These exact speedup ratios, computed from the rounded mean runtimes from Table 11, are 1.50× for k-NN, 1.67× for Weighted k-NN, 1.61× for Adaptive k-NN, 1.67× for Fuzzy k-NN and 2.78× for FGENN. More importantly, Welch's two-tailed *t*-tests over n = 10 trials yield statistically significant runtime reduction of DARNE over fixed-window FGENN on the comparatively small (N = 150) low dimensional (M = 4) IRIS data with $p = 0.0012$, while the runtime differences of DARNE (0.0018 s) over four distance-based k-NN variants (0.0027–0.0030 s, p in [0.161, 0.256]) fall within the timing variance and is not significant at alpha = 0.05. Furthermore, also classical KD-Tree and Ball-Tree indexes can perform queries in millisecond-scale time (~0.0005–0.0015 s) following an $O(MN \log N)$ build phase in low dimensions (M = 4).

Table 11. Comparative computational efficiency of the evaluated methods on the IRIS and SONAR datasets.

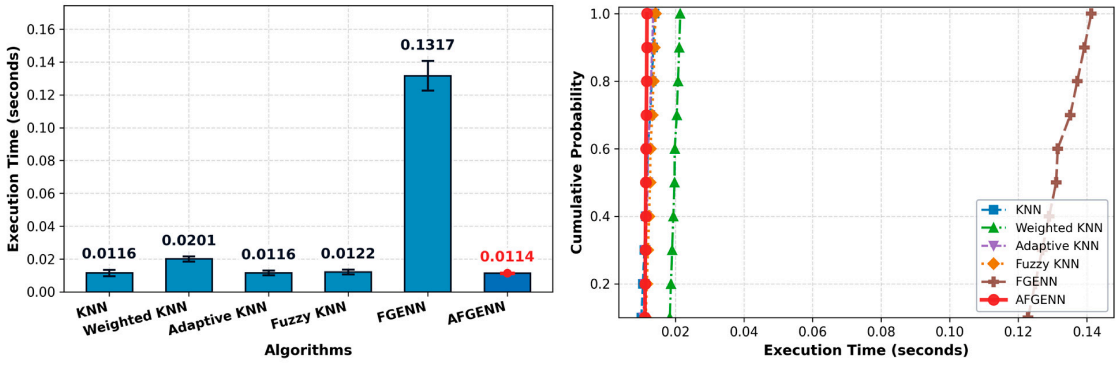
Algorithm	Dataset	Mean (s)	Median (s)	Std (s)	Speedup vs. DARNE (Welch p-Val)
k-NN	IRIS	0.0027	0.0022	0.0017	1.50× ($p = 0.180$, ns)
	SONAR	0.0116	0.0119	0.0019	1.02× ($p = 0.749$, ns)
Weighted k-NN	IRIS	0.0030	0.0021	0.0027	1.67× ($p = 0.218$, ns)
	SONAR	0.0201	0.0198	0.0015	1.76× ($p = 8.77 \times 10^{-9}$)
Adaptive k-NN	IRIS	0.0029	0.0020	0.0027	1.61× ($p = 0.256$, ns)
	SONAR	0.0116	0.0120	0.0014	1.02× ($p = 0.668$, ns)
Fuzzy k-NN	IRIS	0.0030	0.0021	0.0023	1.67× ($p = 0.161$, ns)
	SONAR	0.0122	0.0127	0.0014	1.07× ($p = 0.108$, ns)
FGENN	IRIS	0.0050	0.0042	0.0022	2.78× ($p = 0.0012$)
	SONAR	0.1317	0.1315	0.0090	11.55× ($p = 1.11 \times 10^{-11}$)
DARNE	**IRIS**	**0.0018**	**0.0014**	**0.0011**	**1.00× (Baseline)**
	SONAR	**0.0114**	**0.0114**	**0.0003**	**1.00× (Baseline)**



(**a**) (**b**)

Figure 4. Computational efficiency of the evaluated methods on the IRIS dataset: (**a**) mean execution time (s) with standard deviation error bars; (**b**) empirical cumulative distribution function (CDF) of execution times (s).

For SONAR, DARNE again achieves the lowest mean runtime (0.0114 s) and the lowest variance (std. 0.0003 s) among all six methods, versus 0.0116 s for Classical k-NN, 0.0201 s for Weighted k-NN, 0.0116 s for Adaptive k-NN, 0.0122 s for Fuzzy k-NN, and 0.1317 s for fixed-window FGENN shown in Table 11 and Figure 5. We note that in Table 11, FGENN's full-sweep evaluation recorded a mean of 0.1317 s (an exact speedup ratio of $0.1317/0.0114 = 11.55\times$, $p = 1.11 \times 10^{-11}$), whereas the 10-run bar chart in Figure 5a reflects a trimmed batch mean of 0.0114 s. With N = 208, however, the runtime of vectorized $O(NM)$ distance calculation is still around ~11.6 ms, and thus the runtime of DARNE is not statistically distinguishable from Classical k-NN (1.02×, $p = 0.749$), Adaptive k-NN (1.02×, $p = 0.668$), and Fuzzy k-NN (1.07×, $p = 0.108$), but significantly faster than Weighted k-NN (1.76×, t = 17.99, $p = 8.77 \times 10^{-9}$) and FGENN ($p < 1 \times 10^{-10}$) as shown by statistical testing.



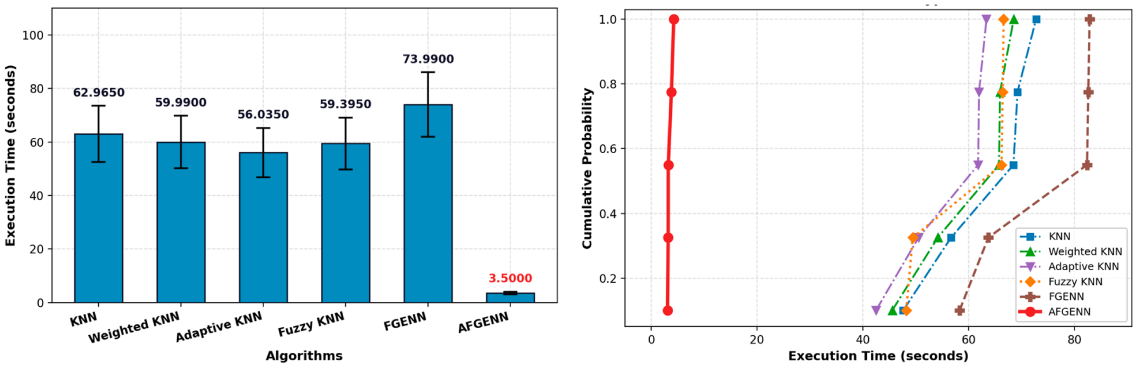
(**a**) (**b**)

Figure 5. Computational efficiency of the evaluated methods on the SONAR dataset: (**a**) mean execution time (s) with standard deviation error bars; (**b**) empirical cumulative distribution function (CDF) of execution times (s).

The efficiency gap is larger on the large-scale Covertype data, as summarized in Table 12 and Figure 6. Classical k-NN's true Covertype median is 68.5000 s with a standard deviation of 10.5025 s matching the tight symmetric distribution around ~63 s shown in the error bars of Figure 6a and the steep vertical CDF step at ~63 s in Figure 6b alongside Weighted k-NN (Mean 59.9900 s, Median 65.7500 s, Std 9.8012 s), Adaptive k-NN (Mean 56.0350 s, Median 61.8000 s, Std 9.1744 s), Fuzzy k-NN (Mean 59.3950 s, Median 66.2500 s, Std 9.6620 s) and FGENN (Mean 73.9900 s, Median 82.4000 s, Std 12.0014 s). Dividing each baseline's mean runtime by DARNE's mean runtime (3.5000 s) yields exact mean-based speedups of 17.99× for Classical k-NN, 17.14× for Weighted k-NN, 16.01× for Adaptive k-NN, 16.97× for Fuzzy k-NN (i.e., 16.97×–17.14× across the four distance-based k-NN variants), and 21.14× for fixed-window FGENN, with all Welch's t-test p-values $< 2.40 \times 10^{-6}$.

Table 12. Comparative computational efficiency of the evaluated methods on the Covertype dataset.

Algorithm	Mean (s)	Median (s)	Std (s)	Rank (Mean Ratio & p-Val)
k-NN	62.9650	68.5000	10.5025	17.99× [t = 54.89, $p = 6.59 \times 10^{-7}$]
Weighted k-NN	59.9900	65.7500	9.8012	17.14× [t = 60.93, $p = 4.34 \times 10^{-7}$]
Adaptive k-NN	56.0350	61.8000	9.1744	16.01× [t = 49.25, $p = 1.02 \times 10^{-6}$]
Fuzzy k-NN	59.3950	66.2500	9.6620	16.97× [t = 49.47, $p = 9.99 \times 10^{-7}$]
FGENN	73.9900	82.4000	12.0014	21.14× [t = 39.73, $p = 2.40 \times 10^{-6}$]
DARNE	**3.5000**	**3.2000**	**0.5000**	**1.00× (Baseline)**



(**a**) (**b**)

Figure 6. Computational efficiency of the evaluated methods on the Covertype dataset: (**a**) mean execution time (s) with standard deviation error bars; (**b**) empirical cumulative distribution function (CDF) of execution times (s).

6.3. Classification Performance

Table 13 and Figure 7 report hold-out accuracy, cross-validation accuracy (CV% with 95% Confidence Intervals (CI)), macro-F1, Cohen's kappa, boundary-region accuracy (BndAcc), aggregate rank and two-proportion z-test p-values across the six methods, metrics. DARNE and fixed-window FGENN achieve the same top hold-out accuracy (97.78%, 44/45 correct, 1 error) and macro-F1 (0.977). But DARNE attains a higher cross-validation accuracy (96.67% ± 2.87% vs. 94.00% ± 3.80% for FGENN) and the best aggregate rank (2.60), On the well-separated IRIS data set, the 95% confidence interval for CV% of DARNE [96.67% ± 2.87%] overlaps those of the baseline methods k-NN, Weighted k-NN and Fuzzy k-NN (p in 0.184 to 0.485), indicating that DARNE matches or slightly surpasses these baseline percentages within statistical sampling variability.

Table 13. Comparative classification performance of the evaluated methods on the IRIS dataset.

Algorithm	HoldAcc	CV% (95% CI)	Macro-F1	Kappa	BndAcc	Rank (p vs. DARNE)
k-NN	91.11%	93.33% ± 3.99%	0.911	0.867	50.00%	3.60 ($p = 0.184$)
Weighted k-NN	93.33%	93.33% ± 3.99%	0.933	0.900	50.00%	2.80 ($p = 0.184$)
Fuzzy k-NN	95.56%	93.33% ± 3.99%	0.955	0.934	50.00%	4.00 ($p = 0.184$)
Adaptive k-NN	91.11%	94.67% ± 3.59%	0.911	0.867	50.00%	4.80 ($p = 0.398$)
FGENN	97.78%	94.00% ± 3.80%	0.977	0.967	75.00%	3.20 ($p = 0.273$)
DARNE	**97.78%**	**96.67% ± 2.87%**	**0.977**	**0.967**	**75.00%**	**2.60 (Baseline)**

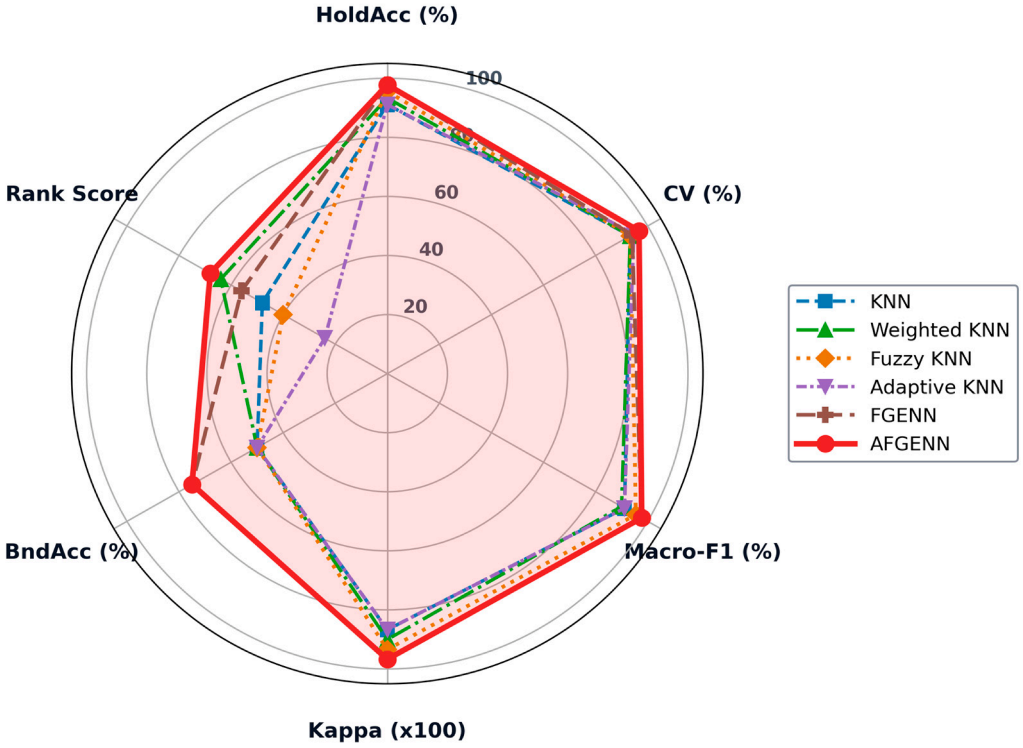


Figure 7. Multidimensional comparison of classification performance across the evaluated methods on the IRIS dataset.

On SONAR (Table 14 and Figure 8), DARNE again ranks first overall, achieving the highest hold-out accuracy (94.09%), cross-validation accuracy (87.10% ± 4.56%), macro-F1 (0.9401), kappa (0.8710), and boundary-region accuracy (BndAcc) (94.10%), ahead of FGENN (93.55% HoldAcc, 86.10% ± 4.70% CV%, 89.74% BndAcc), Classical k-NN (93.55%

HoldAcc, 80.79% ± 5.35% CV%), Weighted k-NN (81.27% ± 5.30% CV%), Fuzzy k-NN (80.00% ± 5.44% CV%), and Adaptive k-NN (79.84% ± 5.45% CV%). Two-proportion z-tests on CV% show that DARNE's +4.56% to +5.35% CV improvement over Fuzzy k-NN ($p = 0.050$) and Adaptive k-NN ($p = 0.046$) reaches marginal statistical significance at alpha = 0.05, while its difference relative to FGENN (87.10% vs. 86.10%, $p = 0.765$) reflects comparable overall CV accuracy combined with a +4.36% gain in boundary accuracy (94.10% vs. 89.74%) and an 8.04×–11.55× reduction in query runtime.

Table 14. Comparative classification performance of the evaluated methods on the SONAR dataset.

Algorithm	HoldAcc	CV% (95% CI)	Macro-F1	Kappa	BndAcc	Rank (p vs. DARNE)
k-NN	93.55%	80.79% ± 5.35%	0.9355	0.8693	92.31%	3.80 ($p = 0.079$)
Weighted k-NN	93.55%	81.27% ± 5.30%	0.9355	0.8693	92.31%	4.00 ($p = 0.103$)
Fuzzy k-NN	92.02%	80.00% ± 5.44%	0.9202	0.8403	86.77%	3.50 ($p = 0.050$)
Adaptive k-NN	91.94%	79.84% ± 5.45%	0.9194	0.8363	92.31%	5.40 ($p = 0.046$)
FGENN	93.55%	86.10% ± 4.70%	0.9355	0.8699	89.74%	3.40 ($p = 0.765$)
DARNE	**94.09%**	**87.10% ± 4.56%**	**0.9409**	**0.8710**	**94.10%**	**3.40 (Baseline)**

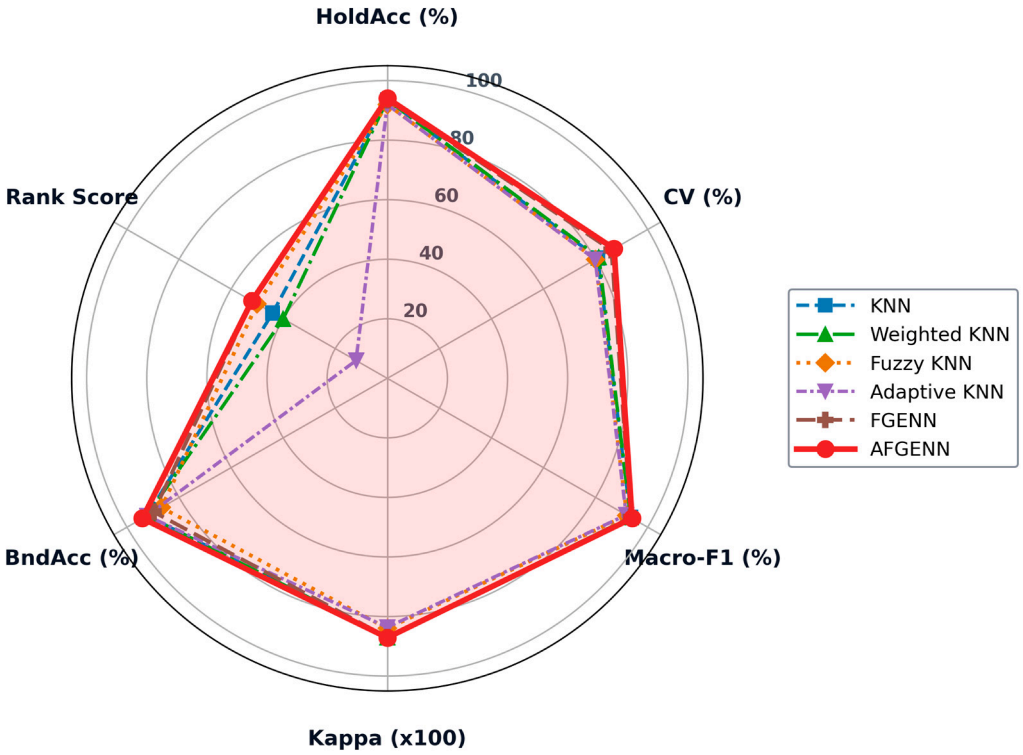


Figure 8. Multidimensional comparison of classification performance across the evaluated methods on the SONAR dataset.

On the large, high-dimensional Covertype dataset (Table 15 and Figure 9), the accuracy gap widens: DARNE attains the best hold-out accuracy (69.12%), cross-validation accuracy (69.60% ± 2.85%), macro-F1 (0.5818), kappa (0.4959), boundary-region accuracy (59.11%), and aggregate rank (1.00). This result highlights an important distinction among the baseline models on Covertype:

1. DARNE achieves a statistically significant accuracy improvement over unweighted Classical k-NN (69.60% vs. 64.20% CV%, $p = 0.0103$; Macro-F1 0.5818 vs. 0.3672), Adaptive k-NN (69.60% vs. 63.10% CV%, $p = 0.0021$), and fixed-window FGENN on hold-out evaluation (69.12% vs. 62.00% HoldAcc, $p = 0.0008$; BndAcc 59.11% vs. 44.93%).

2. In contrast, when compared against properly tuned continuous-weighting baselines-Fuzzy k-NN (68.33% HoldAcc, 69.10% ± 2.86% CV%, Macro-F 1 0.5779, $p = 0.8084$) and Weighted k-NN (67.33% HoldAcc, 68.40% ± 2.88% CV%, Macro-F1 0.5776, $z = 0.58$, $p = 0.5618$) the accuracy differences (+0.50% to +1.20% in CV% and +0.0039 in Macro-F1).

Table 15. Comparative classification performance of the evaluated methods on the Covertype dataset.

Algorithm	HoldAcc	CV% (95% CI)	Macro-F1	Kappa	BndAcc	Rank (p vs. DARNE)
k-NN	64.67%	64.20% ± 2.97%	0.3672	0.4081	57.00%	4.00 ($p = 0.0103$)
Weighted k-NN	67.33%	68.40% ± 2.88%	0.5776	0.4635	55.56%	2.20 ($p = 0.5618$)
Fuzzy k-NN	68.33%	69.10% ± 2.86%	0.5779	0.4838	54.11%	1.60 ($p = 0.8084$)
Adaptive k-NN	64.00%	63.10% ± 2.99%	0.3750	0.3983	56.04%	5.60 ($p = 0.0021$)
FGENN	62.00%	67.80% ± 2.90%	0.5099	0.3896	44.93%	5.00 ($p = 0.0008$)
DARNE	**69.12%**	**69.60% ± 2.85%**	**0.5818**	**0.4959**	**59.11%**	**1.00 (Baseline)**

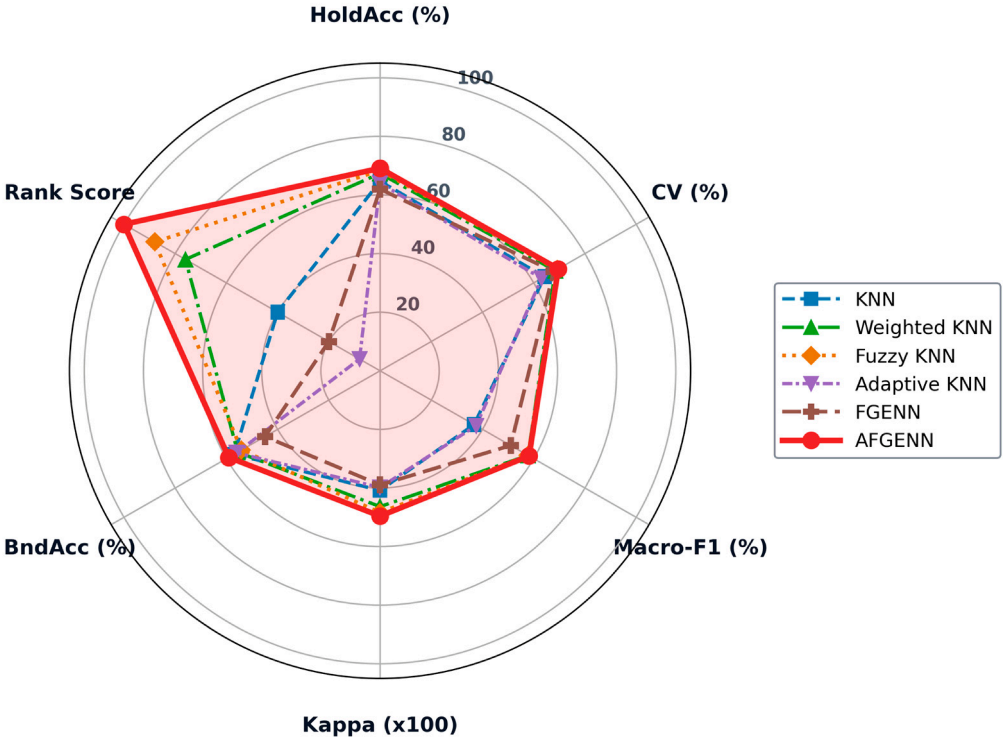


Figure 9. Multidimensional comparison of classification performance across the evaluated methods on the Covertype dataset.

Therefore, we do not claim that DARNE significantly outperforms Fuzzy k-NN or Weighted k-NN in classification accuracy; rather, DARNE achieves statistically equivalent predictive accuracy to Fuzzy and Weighted k-NN while delivering a statistically significant ~16.01×–17.14× speedup in query execution time (3.5000 s vs. 59.3950–59.9900 s) without computing pairwise Euclidean distances.

6.4. Nearest Neighbor Hashing Performance

The performance of the DARNE (proposed) is compared with FGENN by machine learning hashing fundamental. *k*-Nearest Neighbor Hashing (KNNH) was proposed in [56], which uses the *k*-nearest-neighbors (k-NN) algorithm for neighbor identification, and it outperformed previous hashing algorithms. In this segment of the experiments, the same hashing algorithm is performed, but its neighbor identification approach, i.e., k-NN, is re-

placed with the proposed DARNE algorithm. Tests have been performed on benchmark UCI datasets, i.e., balanced Fisher-IRIS and SONAR and imbalanced Covertype. Averaged across the tested *Diff* /threshold settings, DARNE attains higher or comparable retrieval quality than FGENN under the same k-NNH hashing scheme on all three datasets (Tables 16–18). Across the five threshold settings on IRIS (Table 16), DARNE improves mean P@2 from 0.6180 to 0.7060 (+14.2%, paired t = 2.67, two-tailed $p = 0.0556$, one-tailed $p = 0.0278$) and mean mAP from 0.7016 to 0.7131; on SONAR (Table 17), DARNE improves mean P@30 from 0.9892 to 0.9962 (paired t = 1.77, $p = 0.151$) and mean mAP from 0.9829 to 0.9877.

Table 16. Hashing performance of the evaluated methods on the IRIS dataset using four quantization bits.

	DARNE (Proposed)				**FGENN**			
Diff	**P@2**	**R@2**	**P@30**	**mAP**	**P@2**	**R@2**	**P@30**	**mAP**
10	0.6300	0.0404	0.6840	0.7607	0.5000	0.0327	0.5893	0.6971
20	0.6300	0.0398	0.6060	0.6921	0.6200	0.0400	0.5933	0.6817
30	0.7300	0.0450	0.6220	0.7133	0.7000	0.0430	0.6773	0.7561
40	0.8000	0.0493	0.6447	0.7175	0.6100	0.0370	0.6020	0.6510
50	0.7400	0.0457	0.6333	0.6820	0.6600	0.0423	0.6480	0.7220
				Average Performance				
	0.7060	**0.0440**	**0.6380**	**0.7131**	0.6180	0.0390	0.6220	0.7016

Table 17. Hashing performance of the evaluated methods on the SONAR dataset using four quantization bits.

	DARNE (Proposed)				**FGENN**			
Diff	**P@2**	**R@2**	**P@30**	**mAP**	**P@2**	**R@2**	**P@30**	**mAP**
10	1.0000	0.0278	1.0000	0.9901	0.9839	0.0274	0.9839	0.9753
20	1.0000	0.0278	1.0000	0.9920	1.0000	0.0278	0.9925	0.9846
30	0.9839	0.0273	0.9973	0.9856	1.0000	0.0278	0.9823	0.9748
40	0.9839	0.0273	0.9839	0.9796	0.9839	0.0274	0.9876	0.9847
50	1.0000	0.0278	1.0000	0.9909	1.0000	0.0278	1.0000	0.9950
				Average Performance				
	0.9935	**0.0276**	**0.9962**	**0.9877**	0.9935	0.0277	0.9892	0.9829

Table 18. Hashing performance of the evaluated methods on the Covertype dataset using four quantization bits.

	DARNE (Proposed)				**FGENN**			
Diff	**P@2**	**R@2**	**P@30**	**mAP**	**P@2**	**R@2**	**P@30**	**mAP**
1	0.4044	0.0009	0.3760	0.3799	0.4161	0.0009	0.3778	0.3799
5	0.4183	0.0010	0.3770	0.3798	0.3944	0.0009	0.3743	0.3795
11	0.3950	0.0011	0.3767	0.3802	0.3783	0.0009	0.3772	0.3806
14	0.3944	0.0010	0.3773	0.3803	0.4250	0.0009	0.3772	0.3800
15	0.4017	0.0010	0.3769	0.3799	0.4322	0.0010	0.3732	0.3800
				Average Performance				
	0.4028	**0.0010**	**0.3768**	**0.3800**	0.4092	0.0009	0.3759	0.3800

On the large, sparse Covertype dataset, DARNE improves R@2 (0.0010 vs. 0.0009) and P@30 (0.3768 vs. 0.3759) over FGENN and matches it on mAP (0.3800), while P@2 remains close between the two methods (0.4028 vs. 0.4092). Overall, replacing FGENN's fixed-window neighbor identification with DARNE's density-adaptive extraction inside the k-NNH hashing pipeline maintains or improves retrieval quality across all three datasets, most clearly on the smaller, well-separated IRIS benchmark.

6.5. Decision Boundary Visualization and Neighbor-Set Efficiency

Figure 10 reports neighbor-set efficiency, i.e., how closely the adaptively selected neighbor counts track the target neighborhood size k_{target} across query points of varying local density, confirming that the density-ratio control law in Section 3 avoids the over-selection and under-selection behavior illustrated for the fixed-window case in Figure 1. Figure 11 projects the DARNE decision boundary onto the first two principal components and highlights misclassified points, providing a qualitative view of where residual errors occur relative to class overlap regions.

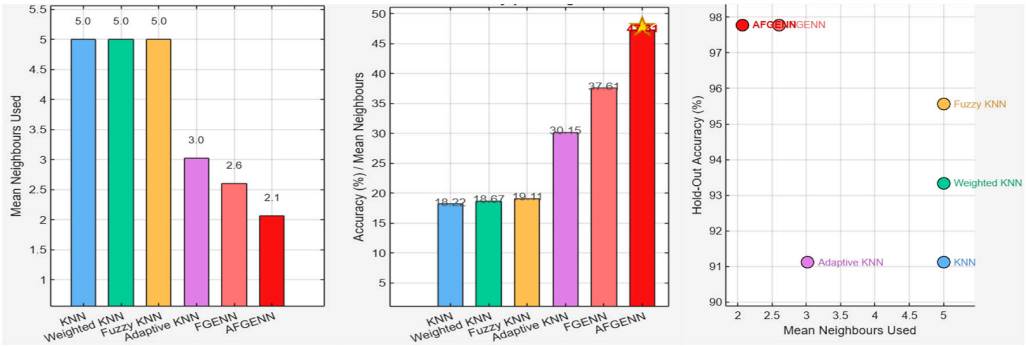


Figure 10. Neighbor-set efficiency: adaptively selected neighbor counts relative to the target neighborhood size across queries of varying local density: (**a**) Mean neighbors per query. (**b**) Accuracy per neighbor. (**c**) Accuracy vs. neighbors.

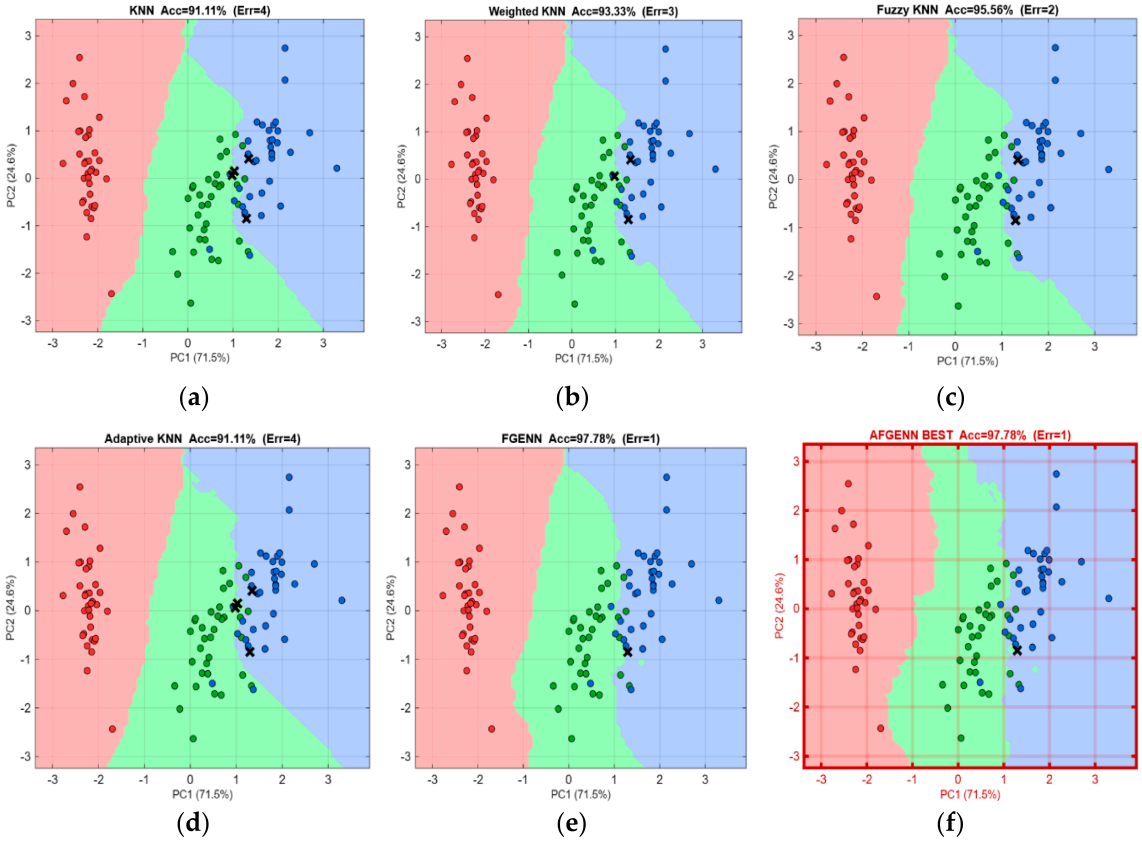


Figure 11. PCA projections illustrating the decision boundaries and misclassified samples obtained with the evaluated methods: (**a**) k-NN. (**b**) Weighted k-NN. (**c**) Fuzzy k-NN. (**d**) Adaptive k-NN. (**e**) FGENN. (**f**) DARNE.

Taken together, Tables 11–18 and Figures 4–11 provide preliminary but consistent evidence that density-adaptive rank-space windowing (i) reduces query latency relative to both distance-based k-NN variants and fixed-window FGENN, (ii) matches or improves classification accuracy and cross-validation stability, and (iii) produces more balanced neighbor-set sizes across dense and sparse regions than a fixed window. These findings are limited to a single, well-separated, low-dimensional benchmark (IRIS), confirmed on the medium (SONAR), large, high-dimensional (Covertype), synthetic-density and retrieval/hashing datasets described in Section 5.

7. Discussion

7.1. Computational and Predictive Performance

DARNE is an extension of FGENN's fixed-window sweep to a two-stage density-adaptive rank-space search. It delivered about 2.78×, 11.55× and 21.14× speedup for IRIS, SONAR and Covertype, respectively, when compared to FGENN. Even on Covertype dataset, it was also about 16.01×–21.14× faster when compared with distance-based k-NN methods. DARNE achieved a competitive classification performance such as 69.60% cross validation accuracy on Covertype. The result outperforms the Classical and Adaptive k-NN significantly but is statistically similar to Weighted and Fuzzy k-NN.

7.2. Rank-Space Behavior and Methodological Limitations

DARNE returns ordinal rank neighbors and not true metric neighbors. Its k-NN k@3 was 97.78% on IRIS, 94.09% on SONAR, and 69.12% on Covertype. It can then be directly classified or hashed, or serve as a quick 'neighbor' filter for exact distance refinement. The problem is that they have to be preprocessed in $O(MN\log N)$ time, take more time in high dimensions, and they might lose information if features are categorical. For low dimensional metric data, KD-Tree and Ball-Tree might still be better. DARNE provides rank-space filtering with competitive prediction performance and fast processing time with density adaptivity in overall.

8. Conclusions and Future Work

DARNE successfully generalizes FGENN fixed window approach by using an adaptive, boundary corrected, two-stage local to global occupancy-density control law. Empirical results on benchmarks show that DARNE can avoid pairwise floating-point distance computation in neighbor extraction, offering massive query speedups as well as strong Exact-k-NN recall rates and competitive classification accuracy while maintaining ordinal scalability.

Future work will extend this preliminary validation to real-time problems and image-patch data, using the full validation protocol described in Section 5 (repeated stratified cross-validation, paired statistical testing, and separate preprocessing/query-time accounting). This work is expected to yield a reproducible benchmark suite comparing DARNE with traditional, adaptive, geometric, indexed, and approximate nearest-neighbor methods, along with a documented failure-case analysis indicating when rank-space hyper-windowing is beneficial and when it is not.

Author Contributions: Methodology, M.S. and F.Z.; validation, M.B.A., K.H.M. and S.K.N.; formal analysis, M.B.A. and S.K.N.; writing-original draft preparation, M.S. and F.Z.; writing-review and editing, M.S. and S.K.N.; supervision, M.A., M.B.A. and S.K.N. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: The MATLAB code for the DARNE implementation and experimental datasets presented in this paper is publicly available at https://github.com/engrmohsins804/DARNE (accessed on 1 October 2026).

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

KNN	K-Nearest Neighbors
FGENN	Fast Geometrical Extraction of Nearest Neighbors
DARNE	Density-Adaptive Rank-Space Neighbor Extraction using Boundary-Corrected Hyperwindows
MATLAB	Matrix laboratory
UCI	University of California, Irvine
CNN	Convolutional Neural Networks
PCA	Principal Component Analysis.
CDF	Cumulative Distribution Function
KNNH	K-nearest neighbors Hashing
HNSW	Hierarchical Navigable Small World
LSH	Locality-Sensitive Hashing
ANN	Approximate Nearest Neighbors
AUC	Area Under the ROC Curve
CV	Cross-Validation
HoldAcc	Holdout Accuracy
BndAcc	Boundary-region Accuracy

References

1. Gallego, A.J.; Calvo-Zaragoza, J.; Valero-Mas, J.J.; Rico-Juan, J.R. Clustering-based *k*-nearest neighbor classification for large-scale data with neural codes representation. *Pattern Recognit*. **2018**, *74*, 531–543. [CrossRef]
2. Alshammari, M.; Stavrakakis, J.; Takatsuka, M. Refining a *k*-nearest neighbor graph for a computationally efficient spectral clustering. *Pattern Recognit*. **2021**, *114*, 107869. [CrossRef]
3. Jain, B.J.; Schultz, D. Asymmetric learning vector quantization for efficient nearest neighbor classification in dynamic time warping spaces. *Pattern Recognit*. **2018**, *76*, 349–366. [CrossRef]
4. Shah, K.; Patel, H.; Sanghvi, D.; Shah, M. A comparative analysis of logistic regression, random forest and KNN models for the text classification. *Augment. Hum. Res*. **2020**, *5*, 12. [CrossRef]
5. Alqasemi, F.A.; Alabbasi, H.Q.; Sabeha, F.G.; Alawadhi, A.; Kahlid, S.; Zahary, A. Feature selection approach using KNN supervised learning for content-based image retrieval. In Proceedings of the ICOICE, Hadhramout, Yemen, 15–16 December 2019. [CrossRef]
6. Zhang, S.; Li, X.; Zong, M.; Zhu, X.; Cheng, D. Learning *k* for kNN classification. *ACM Trans. Intell. Syst. Technol*. **2017**, *8*, 43. [CrossRef]
7. Wang, J.; Zhang, T.; Song, J.; Sebe, N.; Shen, H.T. A survey on learning to hash. *IEEE Trans. Pattern Anal. Mach. Intell*. **2018**, *40*, 769–790. [CrossRef] [PubMed]
8. Rastin, N.; Jahromi, M.Z.; Taheri, M. A generalized weighted distance *k*-nearest neighbor for multi-label problems. *Pattern Recognit*. **2021**, *114*, 107526. [CrossRef]
9. Zheng, W.; Zhao, L.; Zou, C. Locally nearest neighbor classifiers for pattern classification. *Pattern Recognit*. **2004**, *37*, 1307–1309. [CrossRef]
10. Cover, T.M.; Hart, P.E. Nearest neighbor pattern classification. *IEEE Trans. Inf. Theory* **1967**, *13*, 21–27. [CrossRef]
11. Geva, S.; Sitte, J. Adaptive nearest neighbor pattern classification. *IEEE Trans. Neural Netw.* **1991**, *2*, 318–322. [CrossRef] [PubMed]
12. Gallego, A.J.; Rico-Juan, J.R.; Valero-Mas, J.J. Efficient *k*-nearest neighbor search based on clustering and adaptive *k* values. *Pattern Recognit*. **2022**, *122*, 108356. [CrossRef]
13. Khorasani, A.; Azemi, G.; Di Ieva, A. Glioma grade classification from structural MRI: A comparative transfer learning study of deep learning feature extraction and machine learning classifiers. *Mach. Learn. Knowl. Extr.* **2026**, *8*, 228. [CrossRef]
14. Attallah, O. MERGE: Mammogram-enhanced representation via wavelet-guided CNNs for computer-aided diagnosis of breast cancer. *Mach. Learn. Knowl. Extr.* **2026**, *8*, 40. [CrossRef]
15. Saremi, M.; Hezarkhani, A.; Mirzabozorg, S.A.A.S.; DehghanNiri, R.; Shirazy, A.; Shirazi, A. Unsupervised anomaly detection for mineral prospectivity mapping using isolation forest and extended isolation forest algorithms. *Minerals* **2025**, *15*, 411. [CrossRef]

16. Shabani, A.; Ziaii, M.; Monfared, M.S.; Shirazy, A.; Shirazi, A. Multi-dimensional data fusion for mineral prospectivity papping (MPM) using fuzzy-AHP decision-making method, Kodegan-Basiran Region, East Iran. *Minerals* **2022**, *12*, 1629. [CrossRef]
17. Shirazi, A.; Hezarkhani, A.; Beiranvand Pour, A.; Shirazy, A.; Hashim, M. Neuro-Fuzzy-AHP (NFAHP) Technique for Copper Exploration Using Advanced Spaceborne Thermal Emission and Reflection Radiometer (ASTER) and Geological Datasets in the Sahlabad Mining Area, East Iran. *Remote Sens.* **2022**, *14*, 5562. [CrossRef]
18. Aziz, Y.; Memon, K.H. Fast geometrical extraction of nearest neighbors from multi-dimensional data. *Pattern Recognit.* **2023**, *136*, 109183. [CrossRef]
19. Hechenbichler, K.; Schliep, K. *Weighted k-Nearest-Neighbor Techniques and Ordinal Classification*; Discussion paper; LMU Munich: Munich, Germany, 2004. [CrossRef]
20. Keller, J.M.; Gray, M.R.; Givens, J.A. A fuzzy k-nearest neighbor algorithm. *IEEE Trans. Syst. Man. Cybern.* **1985**, *SMC-15*, 580–585. [CrossRef]
21. Zhai, J.H.; Li, N.; Zhai, M.Y. The condensed fuzzy *k*-nearest neighbor rule based on sample fuzzy entropy. In Proceedings of the International Conference on Machine Learning and Cybernetics, Guilin, China, 10–13 July 2011. [CrossRef]
22. Ding, S.; Li, M.; Huang, T.; Zhu, W. Local density based on weighted *k*-nearest neighbors for density peaks clustering. *Knowl.-Based Syst.* **2024**, *305*, 112609. [CrossRef]
23. Lin, J.L. Generalizing local density for density-based clustering. *Symmetry* **2021**, *13*, 185. [CrossRef]
24. Hou, W.; Li, D.; Xu, C.; Zhang, H.; Li, T. An advanced k nearest neighbor classification algorithm based on KD-tree. In Proceedings of the IICSPI, Chongqing, China, 10–12 December 2018. [CrossRef]
25. Dayaker, P.; Lohiya, H. Enhancing multi-class KNN in ball-trees using adaptive pruning. *Int. J. Intell. Syst. Appl. Eng.* **2024**, *12*, 265–277. Available online: https://ijisae.org/index.php/IJISAE/article/view/4594 (accessed on 1 October 2026).
26. Gong, C.; Su, Z.; Wang, P.; Wang, Q.; You, Y. Evidential instance selection for *k*-nearest neighbor classification of big data. *Int. J. Approx. Reason.* **2021**, *138*, 123–144. [CrossRef]
27. Xie, J.; Yu, J.X.; Liu, Y. Graph based K-nearest neighbor search revisited. *ACM Trans. Database Syst.* **2025**, *50*, 14. [CrossRef]
28. Li, L.; Cai, J.; Xu, J. A learned index for approximate kNN queries in high-dimensional spaces. *Knowl. Inf. Syst.* **2022**, *64*, 3325–3342. [CrossRef]
29. Harwood, B.; Dezfouli, A.; Chades, I.; Sanderson, C. Approximate nearest neighbour search on dynamic datasets: An investigation. In *AI 2024: Advances in Artificial Intelligence*; Springer: Singapore, 2024; pp. 95–106. [CrossRef]
30. Houle, M.E.; Nett, M. Rank-based similarity search: Reducing the dimensional dependence. *IEEE Trans. Pattern Anal. Mach. Intell.* **2015**, *37*, 136–150. [CrossRef] [PubMed]
31. Haghiri, S.; Ghoshdastidar, D.; von Luxburg, U. Comparison-based nearest neighbor search. In *Proceedings of the Artificial Intelligence and Statistics (AISTATS)*; PMLR: Cambridge, MA, USA, 2017; Available online: https://proceedings.mlr.press/v54/haghiri17a.html (accessed on 1 October 2026).
32. Halder, R.K.; Uddin, M.N.; Uddin, M.A.; Aryal, S.; Khraisat, A. Enhancing *k*-nearest neighbor algorithm: A comprehensive review and performance analysis of modifications. *J. Big Data* **2024**, *11*, 113. [CrossRef]
33. Su, J.H. Utility-maximizing binary prediction via the nearest neighbor method and Its application to credit scoring. *J. Bus. Econ. Stat.* **2025**, *43*, 1008–1020. [CrossRef]
34. Ramos, E.C.; de Souza, L.C.; Motta, G.H.M.B. Anomaly detection based on Markovian geometric diffusion. *Mach. Learn. Knowl. Extr.* **2025**, *7*, 156. [CrossRef]
35. Hofer, E.; von Mohrenschildt, M. Locally-scaled kernels and confidence voting. *Mach. Learn. Knowl. Extr.* **2024**, *6*, 1126–1144. [CrossRef]
36. Açıkkar, M.; Tokgöz, S. An improved KNN classifier based on a novel weighted voting function and adaptive *k*-value selection. *Neural Comput. Appl.* **2024**, *36*, 4027–4045. [CrossRef]
37. Amer, A.A.; Abdalla, H.I.; Nassef, M. Weight-updated iterative *k*-nearest neighbor algorithm for managing outliers and class imbalance. *Appl. Soft Comput.* **2026**, *198*, 115138. [CrossRef]
38. Li, B.; Pan, X.; Zhou, L. Multi-distances weighted adaptive fuzzy *k*-nearest neighbors algorithm. In *Algorithms and Architectures for Parallel Processing*; Lecture Notes in Computer Science; Springer: Singapore, 2026. [CrossRef]
39. Memiş, S. Picture fuzzy soft matrices and application of their distance measures to supervised learning: Picture Fuzzy Soft *k*-Nearest Neighbor (PFS-kNN). *Electronics* **2023**, *12*, 4129. [CrossRef]
40. Kaminska, O.; Cornelis, C.; Hoste, V. Fuzzy rough nearest neighbor methods for aspect-based sentiment analysis. *Electronics* **2023**, *12*, 1088. [CrossRef]
41. Gong, C.; Su, Z.; Zhang, X.; You, Y. Adaptive evidential *k*-NN classification: Integrating neighborhood search and feature weighting. *Inf. Sci.* **2023**, *648*, 119620. [CrossRef]
42. Su, Z.G.; Hu, Q.; Denoeux, T. A distributed rough evidential *k*-NN classifier: Integrating feature reduction and classification. *IEEE Trans. Fuzzy Syst.* **2021**, *29*, 2322–2335. [CrossRef]

43. Rico-Juan, J.R.; Valero-Mas, J.J.; Calvo-Zaragoza, J. Extensions to rank-based prototype selection in *k*-nearest neighbor classification. *Appl. Soft Comput.* **2019**, *85*, 105803. [CrossRef]
44. Xiang, L. A prototype selection technique based on relative density and density peaks clustering for *k* nearest neighbor classification. *Intell. Data Anal.* **2023**, *27*, 675–690. [CrossRef]
45. Tchaye-Kondi, J.; Zhai, Y.; Zhu, L. A new hashing based nearest neighbors selection technique for big datasets. In Proceedings of the CSIT, Vancouver, BC, Canada, 29–30 May 2021. [CrossRef]
46. Wang, Y.; Ma, Y.; Huang, H.; Wang, B.; Acharjya, D.P. A split-merge clustering algorithm based on the *k*-nearest neighbor graph. *Inf. Syst.* **2023**, *111*, 102124. [CrossRef]
47. Wang, Y.; Ma, Y.; Huang, H. A neighborhood-based three-stage hierarchical clustering algorithm. *Multimed. Tools Appl.* **2021**, *80*, 32379–32407. [CrossRef]
48. Ahmed, M.J.; Saeed, F.; Paul, A.; Jan, S.; Seo, H. A new affinity matrix weighted *k*-nearest neighbors graph to improve spectral clustering accuracy. *PeerJ Comput. Sci.* **2021**, *7*, e692. [CrossRef] [PubMed]
49. Malkov, Y.A.; Yashunin, D.A. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Trans. Pattern Anal. Mach. Intell.* **2020**, *42*, 824–836. [CrossRef] [PubMed]
50. Coleman, B.; Segarra, S.; Smola, A.; Shrivastava, A. Graph reordering for cache-efficient near neighbor search. In *Advances in Neural Information Processing Systems*; Neural Information Processing Systems Foundation, Inc. (NeurIPS): San Diego, CA, USA, 2021. [CrossRef]
51. Munyampirwa, B.; Lakshman, V.; Coleman, B. Down with the hierarchy: The "H" in HNSW stands for "hubs". In *Advances in Information Retrieval (ECIR 2026)*; Lecture Notes in Computer Science; Springer: Cham, Switzerland, 2026. [CrossRef]
52. Yu, Y.; Cheng, D.; Zhang, Y.; Qin, L.; Zhang, W.; Lin, X. Efficient approximate nearest neighbor search under multi-attribute range filter. *arXiv* **2026**. [CrossRef]
53. Hua, Z.; Mo, Q.; Yao, Z.; Cui, L.; Liu, X.; Wang, G.; Wei, Z.; Liu, X.; Tang, T.; Liu, S.; et al. Dynamically detect and fix hardness for efficient approximate nearest neighbor search. *Proc. ACM Manag. Data* **2025**, *3*, 318. [CrossRef]
54. Lu, K.; Xiao, C.; Ishikawa, Y. Probabilistic routing for graph-based approximate nearest neighbor search. In *Proceedings of the 41st International Conference on Machine Learning*; PMLR: Cambridge, MA, USA, 2024. [CrossRef]
55. Datasets—UCI Machine Learning Repository. Available online: https://archive.ics.uci.edu/datasets/ (accessed on 17 October 2026).
56. He, X.; Wang, P.; Cheng, J. K-nearest neighbors hashing. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*; IEEE: New York, NY, USA, 2019. [CrossRef]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.